

slingpuck

Sim2real2sim reinforcement learning for an SO-101 arm playing Fast Sling Puck
Project report: what is done, how the code works, what is still to do

08 October 2026 · git commit `2e4f826` (16 commits)
38 source files, 5117 lines of Python · 127 tests, all passing
Milestones M1–M4 done; M5 not started

Contents

1. Summary
2. Status and history
3. Key decisions and findings
4. How the system fits together
5. Configuration files
6. Figures explained (training curves, evaluation curves, models, viewers)
7. Evaluation results (tables)
8. Code reference: every file, class and function
9. Tests
10. How to run
11. Constraints and difficulties faced
12. What can still be improved
13. What is still to do
14. Glossary

1. Summary

The project builds a training and evaluation workspace for an SO-101 arm (6-DOF, Feetech STS3215 servos) that plays tabletop sling puck on a moopok Fast Sling Puck board (size M), in three phases: **Phase 1** goalkeeper (block pucks at the gate), **Phase 2** targeted shot (sling a puck through the gate), and **Phase 3** a match against an opponent (choose when to block, sling or hold).

Done (M1–M4):

- A versioned configuration system: every unmeasured value is marked as a placeholder, and strict mode refuses to use it for real results.
- Physics and sensing models: a fast 2D puck sim, a servo model, a camera model and a Kalman tracker, a band model with a closed hysteresis loop, arm deflection, and a MuJoCo backend with the real SO-101 model.
- Three Gymnasium environments. Phases 1 and 2 are trained with an asymmetric actor-critic PPO (3 seeds each) and evaluated with confidence intervals. Phase 3 has a scripted opponent and rule-based baselines.
- A system identification (sysid) pipeline, tested on synthetic logs: it fits the band, servo, compliance, friction, restitution and energy transfer, writes a new parameter version, and checks sim-vs-real parity.
- Viewers (2D and 3D MuJoCo), a README, and 127 tests.

Main open points: real measurements (servo step response, band force test, shot logs, human opponent timing), the real **scoring rules**, and milestone M5 (selector training, ablations, vulnerability window, ONNX export and Jetson latency).

2. Status and history

Milestone	Content	Status
M1	Physics, servo, camera, tracker, config versioning, unit tests	Done
M2	Phase 1 env, asymmetric PPO, save-rate eval, viewers	Done (3 seeds)
—	MuJoCo backend for Phase 1, 3D viewer, sim-to-sim eval (user request)	Done
M3	Phase 2 env, band and deflection models, sysid with synthetic logs, parity eval	Done (3 seeds)
M4	Opponent model, Phase 3 match env, frozen primitives, rule-based baselines	Done
M5	Selector training, ablations, vulnerability window, ONNX, latency	Not started

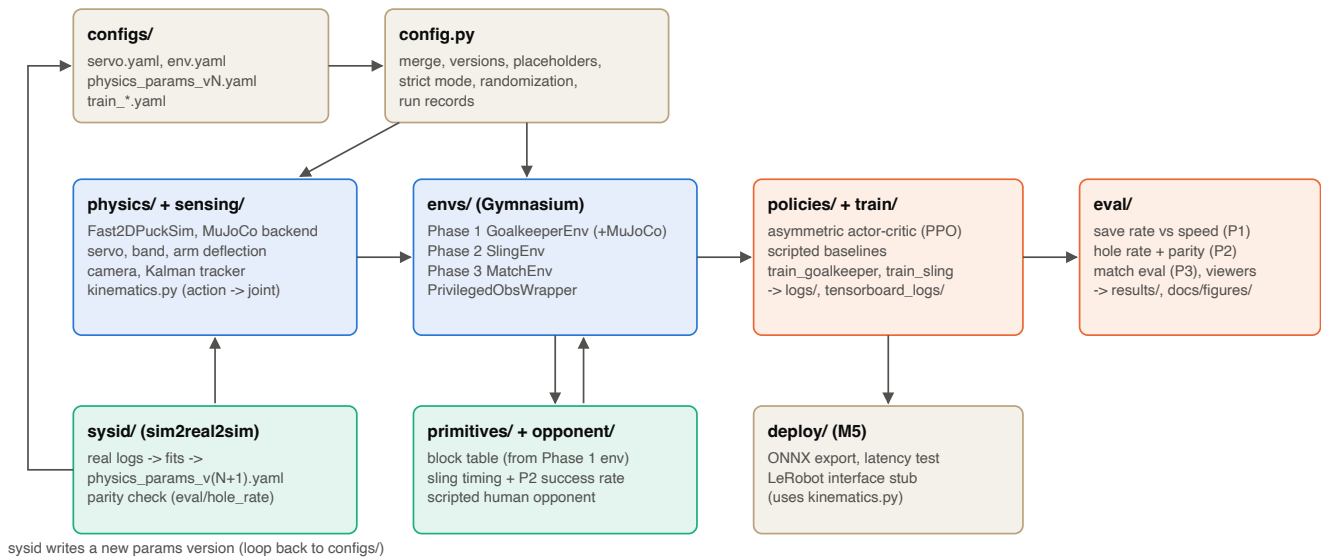
All milestones wait for the user's review. Git history (oldest first):

Commit	Message
10d29f9	Baseline snapshot before M1 rework
c20b8ea	M1: config versioning, servo/camera/tracker fixes, fast 2D puck sim, tests
7810cbe	Config: user-measured puck mass (27 g) and gate width (~1.25 in)
36e0faa	M2: goalkeeper env on Fast2DPuckSim, asymmetric PPO, save-rate eval
69ce734	M2: recovery task (paddle locked until launch + release delay), grid eval
74d72f9	Add top-down slow-motion goalkeeper viewer
d8b2414	README: full project documentation (setup, configs, models, Phase 1, results, open items)
88ebf85	Fix trained-run loading across module paths; document how to run
6f285e8	WIP: MuJoCo goalkeeper backend, 3D viewer, eval backend option
c2947e0	MuJoCo goalkeeper: calibrated contacts, planar puck, tests, sim-to-sim results
d354cd8	M3: pre-stretched band with closed hysteresis loop, arm deflection, Phase 2 sling env
90506f0	Band release: puck that cannot beat friction stays behind the band (no launch)
acb49f4	Sysid: log schemas, synthetic logs, band/servo/shot fits
3a1b0d0	M3: run_sysid, hole-rate and parity eval, schema docs, example logs, results
2c192eb	M4: opponent model, event-level match env, frozen primitives, baselines, match eval
2e4f826	Add project report PDF (status, decisions, code reference, results, to-do)

3. Key decisions and findings

1. **The original code had bugs that made its results invalid.** Opponent shots flew away from the gate (sign error); goals during a sling recovery were not counted; the sim mapped action 1.0 to 0.19 rad and the robot stub to 0.5 rad; a stale camera frame was used again and again; the camera fell back to ground truth when no frame came; random numbers were not seeded. All are fixed and covered by tests.
2. **Board geometry from the listing (size M).** Outer 14.4×8.5 in; inner play area 0.335×0.185 m; puck 1.19×0.4 in. The gate is only about 1.5 mm wider than the puck (user: "a little over 1.19 in"). The user-measured puck mass of 27 g gives a density of 3.7 g/cm^3 (wood is 0.5–0.75), so it may be the mass of 6 pucks ($6 \times 4.5 \text{ g}$); this needs a check.
3. **Phase 1 is a recovery task (user decision).** Because the gate is so narrow, a paddle settled at the gate blocks everything. Phase 1 therefore measures the *time to cover* the gate when the arm starts away from it, which is the input for the Phase 3 vulnerability window.
4. **PPO learned a servo trick in 2D** (command the far side of the range, so the first-order lag keeps the paddle at full speed). It beat "go to center" in 2D (e.g. 85% vs 78% at 1.9–2.3 m/s), but **not in MuJoCo**, where the torque-limited arm starts more slowly (37 ms vs 81 ms to move halfway). The 2D servo model is the weakest part of Phase 1; the real step response must be measured.
5. **The band launches almost straight (Phase 2).** With a pre-stretched band and pulls of a few cm against an 18.5 cm span, the pull angle barely steers the puck; a shot scores only if the puck leaves the band within ± 3 mm of the gate center. The user chose **placement**: the arm slides the puck along the band, then pulls.
6. **For a still puck, average the camera frames.** The constant-velocity Kalman filter gave ~ 2 mm error on a still puck and made the correction step harmful; the frame mean gives ~ 0.7 mm.
7. **Sysid works on synthetic data after four fixes.** Band stiffness, exponent and natural length trade off (measure the natural length with a ruler); a deadband needs a grid search; frame selection by noisy positions biased speeds 8% low; a bounce between two frames contaminated a segment. All shot parameters are now within about one standard error over 6 synthetic data sets. Parity: the uncalibrated sim is 9.5 ms too fast in transit time; the fitted one agrees ($+0.7$ ms).
8. **Phase 3 is an event-level match over measured primitive tables** (70,000 decisions/s). With the placeholder timings and the "first empty side wins" rule, the match is a **throughput race**: the plan's baseline ("sling only when the opponent reloads") loses, because a robot sling cycle (~ 3.1 s) is much longer than a human reload (~ 0.6 s); the hand tell helps only a little. The answer to the research question depends on the real scoring rules and the real robot and human timings.

4. How the system fits together



The configuration flows into the physics and sensing models; the environments combine them into Gymnasium tasks; the policies are trained on the environments and evaluated by eval/. Phase 3 uses the Phase 1 and Phase 2 skills as *frozen primitives*: their measured performance tables. sysid closes the sim2real2sim loop: real logs produce a new physics parameter version, which the sims load.

Board frame

Origin at the board center; x across the width, y along the length. The agent half is $y < 0$, the opponent half $y > 0$, and the center divider with the gate is at $y = 0$. Pucks launched by the agent move toward $+y$.

Fidelity levels

Sim	Used for	Speed
Fast2DPuckSim + ServoModel	Phase 1 and 2 training and eval	~2,300 control steps/s per process
MuJoCo (board, puck, SO-101 arm)	Phase 1 sim-to-sim check, 3D viewing	~180 episodes/s
MatchEnv (event level, primitive tables)	Phase 3 selector	~70,000 decisions/s

5. Configuration files

`load_config(params_version)` merges, in this order: `servo.yaml` → `physics_params_v<N>.yaml` → `env.yaml`.

File	Content
<code>configs/physics_params_v0.yaml</code>	Measured or fitted physics, version 0 (uncalibrated): board, puck, band, arm compliance, robot mounting and paddle, camera, servo overrides, and the placeholder list. Each value has a source tag: [spec] listing text, [image] listing image, [user] user measurement, [derived] computed, TODO unknown.
<code>configs/servo.yaml</code>	Servo model: max rate (300 deg/s), lag time constant, deadband, command latency (all placeholders).
<code>configs/env.yaml</code>	Design choices: sim timing (physics 1 ms, control 30 Hz, MuJoCo 0.5 ms), tracker tuning, <code>goalkeeper</code> task (pan range, speeds, pre-launch, recovery lock), <code>sling</code> task (ranges, noise, correction, reward shaping), <code>opponent</code> (cycle timing, accuracy, tell), <code>match</code> (pucks, robot timings, block table grid), <code>scoring</code> (placeholder rules) and <code>randomization</code> (domain randomization ranges).
<code>configs/train_goalkeeper.yaml</code> , <code>train_sling.yaml</code>	PPO hyperparameters, policy type (asymmetric or mlp), number of envs, steps and seeds.

33 values are still placeholders in v0:

- `servo.max_rate_deg_s`
- `servo.lag_tau_s`
- `servo.deadband_deg`
- `servo.command_latency_s`
- `board.wall_thickness_m`
- `board.divider_thickness_m`
- `board.gate_width_m`
- `board.restitution`
- `board.paddle_restitution`
- `puck.radius_m`
- `puck.thickness_m`
- `puck.friction_kinetic`
- `puck.restitution`
- `band.anchor_span_m`
- `band.band_offset_from_end_wall_m`
- `band.natural_length_m`
- `band.stiffness_k_n_m`
- `band.stiffness_exponent`
- `band.hysteresis_loss_factor`
- `band.energy_transfer`
- `band.max_pull_m`
- `arm.deflection_compliance_m_per_n`
- `robot.base_xy_m`
- `robot.pan_zero_offset_rad`
- `robot.paddle_width_m`
- `robot.paddle_thickness_m`
- `robot.paddle_height_m`
- `robot.paddle_mass_kg`
- `robot.base_z_m`
- `board.wall_height_m`
- `camera.latency_s`
- `camera.noise_std_m`
- `camera.dropout_prob`

Other data: `data/real_logs/schema.md` (how to record the sysid logs), `data/real_logs/synthetic_example/` (small synthetic logs with `truth.yaml`), `assets/mujoco/robotstudio_so101/` (SO-101 MuJoCo model and meshes), `assets/urdf/so101.urdf` (not used yet), `cache/` (primitive tables), `logs/`, `tensorboard_logs/`, `results/` (outputs, not in git).

6. Figures explained

Every figure made during the project, in the order of the work: training curves, evaluation curves, model checks and viewer frames. For each: what it shows, how to read it, and what it tells us.

Figure 1. Phase 1 training curves (goalkeeper PPO, 3 seeds, 1M steps each)

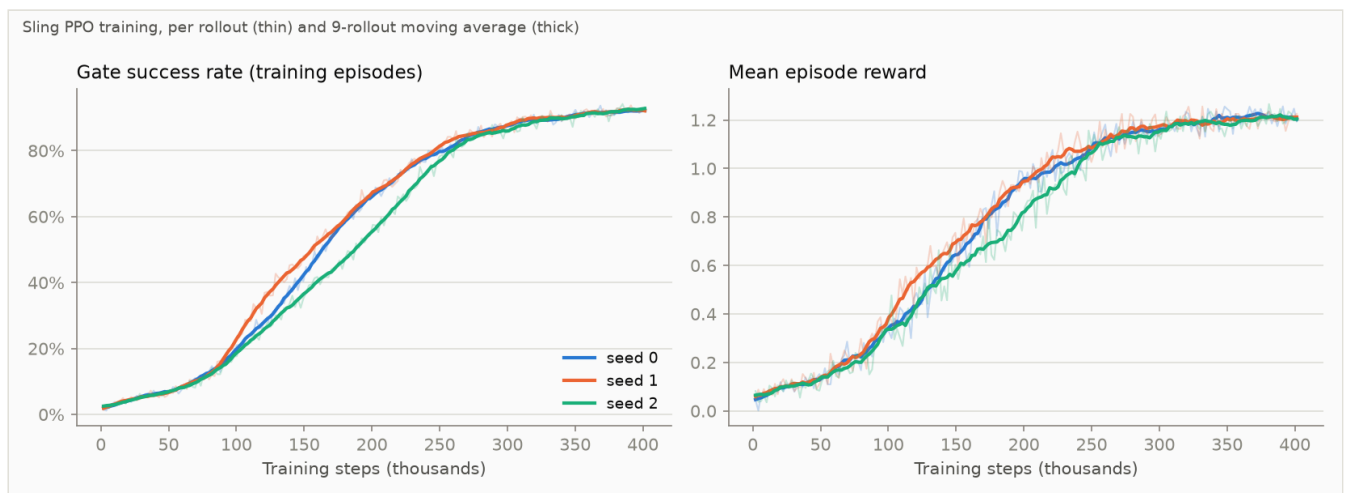


What it shows. The save rate and the mean episode reward of the training episodes, logged once per PPO rollout (4,096 steps). Thin lines are the raw values; thick lines are a 9-rollout moving average. One color per seed.

How to read it. Higher is better. The training episodes include exploration noise and domain randomization, so they are lower than the deterministic eval. The reward is +1 save, -1 goal, minus a small smoothness penalty.

What we see. All seeds learn in the first ~50k steps: the save rate goes from about 60% to about 84% and then stays level for the rest of the 1M steps. The three seeds are almost the same, which agrees with the eval finding that all seeds learned the same saturated 'command the far side' policy. The plateau is set by physics (fast shots from a far start cannot be reached), not by training. Most of the 1M steps were not needed.

Figure 2. Phase 2 training curves (sling PPO, 3 seeds, 400k steps each)



What it shows. Gate success rate and mean episode reward of the training episodes per rollout (2,048 steps).

How to read it. The reward is +1 for a gate pass plus up to 0.3 shaping for a near miss, so the maximum is about 1.3.

What we see. An S-shaped learning curve: almost no success for the first ~80k steps (the gate is a +-3 mm target), then a steady rise as the policy learns to slide the puck to the center, and a plateau at about 92% after ~350k steps. Seed 2 learns about 20k steps later but reaches the same level. While successes are rare, the near-miss shaping term gives most of the reward and so the early learning signal.

Figure 3. Training curve of the original code (before the rework)

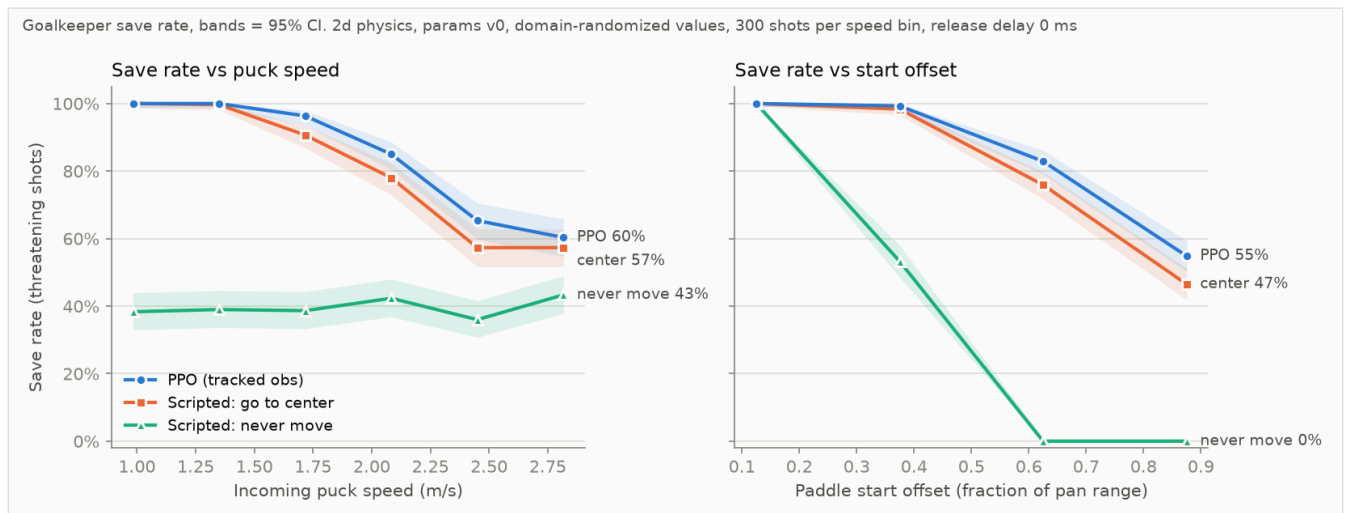


What it shows. Mean episode reward of the single goalkeeper PPO run that existed when the project was handed over.

How to read it. This run used the original env (0.6 x 0.3 m board, 5 cm gate, stale camera frames, a ground-truth fallback, and a paddle 'save' even for shots that would miss).

What we see. The reward reaches about 0.97, which looks very good, but the env had the bugs listed in section 3, so the number is not valid. It is kept for comparison only.

Figure 4. Phase 1 evaluation on 2D physics

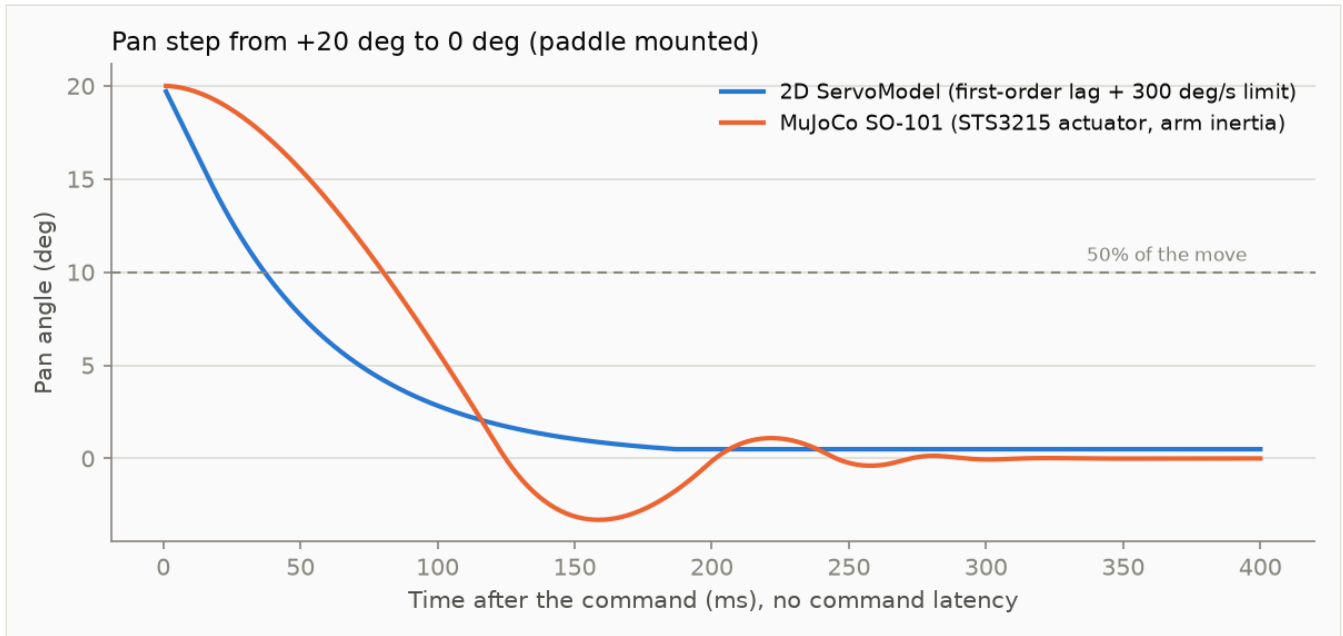


What it shows. Left: save rate vs incoming puck speed (6 bins, 300 threatening shots per bin). Right: save rate vs the paddle start offset (fraction of the pan range). Blue: PPO (3 seeds), orange: scripted 'go to center', aqua: 'never move'. Bands are 95% CIs.

How to read it. Every policy faces the same shots (paired seeds). The 'never move' line is the lower bound: it saves only shots that hit a paddle that happens to start near the center.

What we see. All policies save everything below ~1.5 m/s. Above that the save rate falls because the paddle cannot reach the gate in time from far starts (right panel: 'never move' drops to 0% beyond half the range). PPO is 3-8 points better than 'go to center' at 1.5-3.0 m/s because it commands the far side of the range, which makes the first-order lag of the 2D servo model move faster.

Figure 5. Pan step response: 2D servo model vs MuJoCo arm

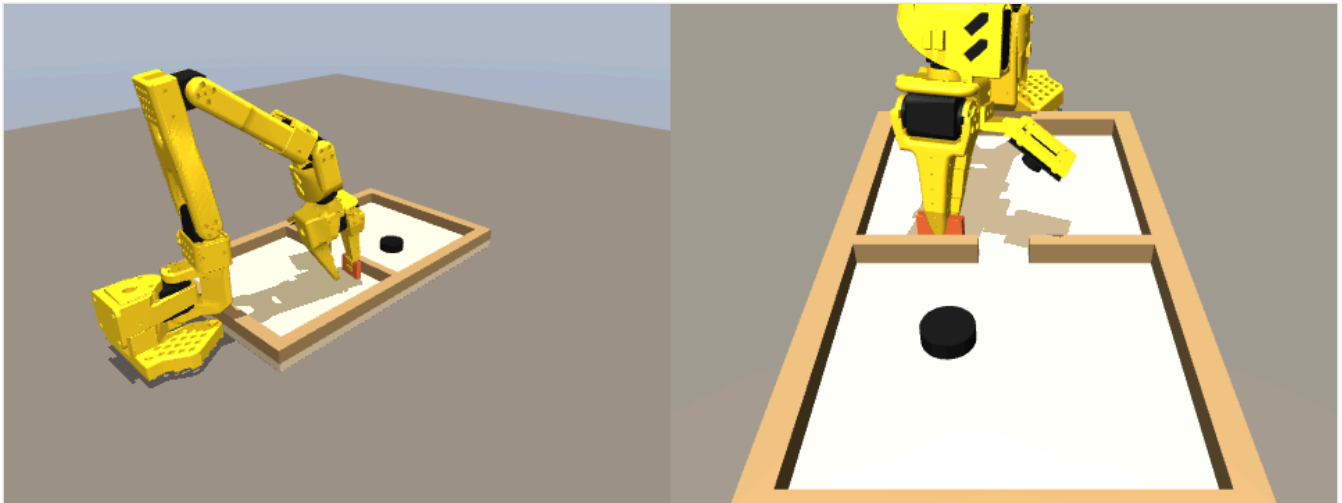


What it shows. The pan angle after a command from +20 deg to 0 deg (no command latency) in the 2D ServoModel (blue) and on the MuJoCo SO-101 with the paddle (orange). The dashed line is 50% of the move.

How to read it. The time to cross the dashed line is the useful reaction time for a goalkeeper.

What we see. The 2D model starts at full speed at once (first-order lag) and reaches 50% in about 37 ms; it stops 0.5 deg short because of the deadband. The MuJoCo arm accelerates slowly (torque limit and inertia), reaches 50% in about 81 ms, overshoots to about -3 deg and rings for ~150 ms. This difference explains the sim-to-sim gap.

Figure 6. The MuJoCo scene

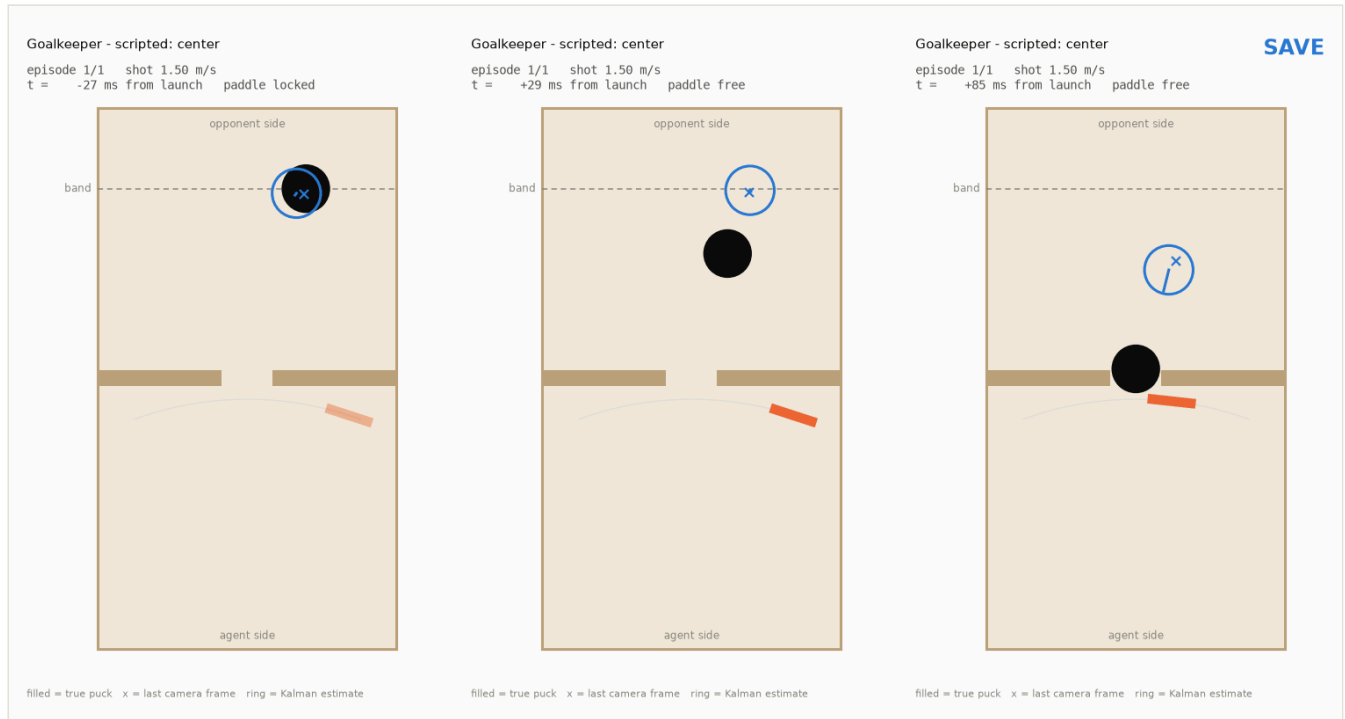


What it shows. Side and gate camera views of the SO-101 holding the paddle in front of the gate, on the real-size board.

How to read it. Orange box: the paddle (a custom mount on the gripper). Black: the puck.

What we see. The scene is built from the same config as the 2D sim, so the board, gate and paddle arc are identical.

Figure 7. 2D viewer frames (one Phase 1 episode, slow motion)

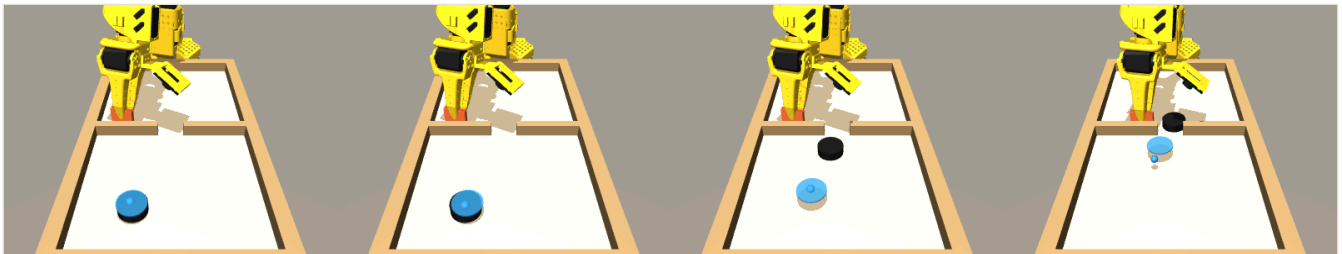


What it shows. Three frames of the top-down viewer: before the launch (paddle locked, faded), during the flight, and the save. Filled black: true puck; blue x: last camera frame; blue ring with line: Kalman estimate and velocity.

How to read it. Time is shown relative to the launch. The shot is 1.5 m/s with the paddle starting at 90% of its range.

What we see. The paddle reaches the gate just in time and saves. The Kalman ring is far behind the puck at +29 and +85 ms: with 50 ms latency only one or two frames after the launch have arrived, and the constant-velocity filter reacts slowly to a launch from rest.

Figure 8. 3D MuJoCo viewer frames (the same shot)

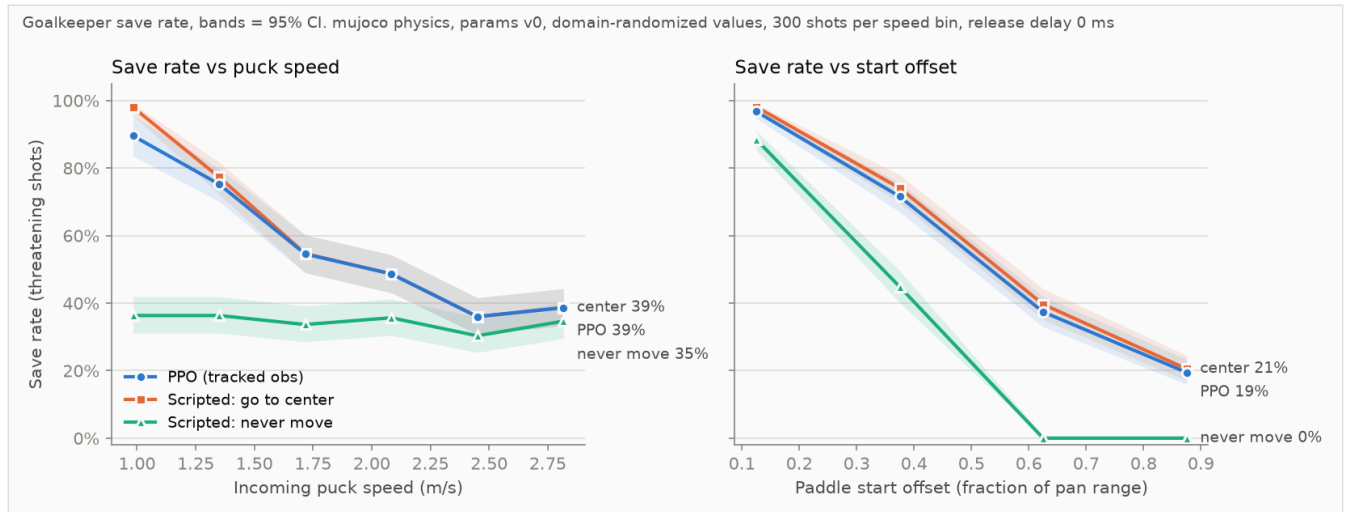


What it shows. Four frames from the gate camera for the same shot as above, on MuJoCo physics.

How to read it. The blue disk is the Kalman estimate; the black puck is the truth.

What we see. With the slower MuJoCo arm the same shot is a goal (the puck passes the gate in the last frame), while the 2D sim saves it. One example of the sim-to-sim gap.

Figure 9. Phase 1 evaluation on MuJoCo physics (sim-to-sim)

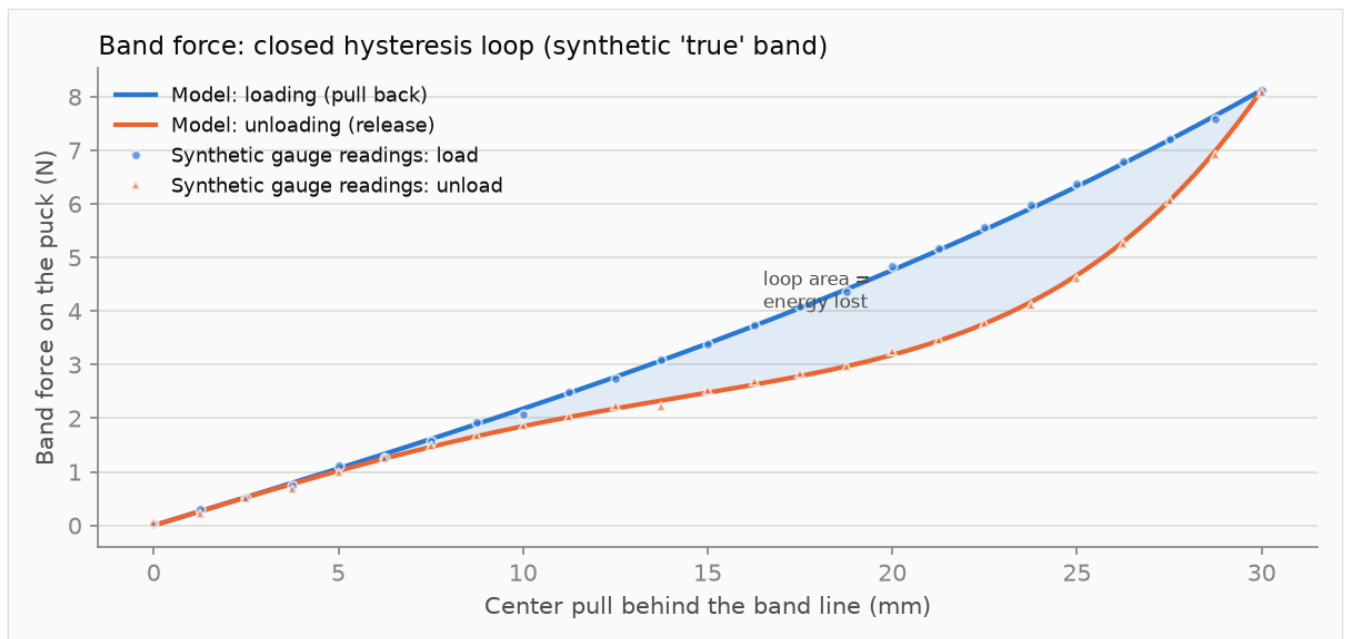


What it shows. The same eval as on 2D physics, with the same trained policies, on MuJoCo.

How to read it. Compare each line with the 2D figure.

What we see. All policies lose 20-40 points above 1.2 m/s. PPO is never better than 'go to center' and is worse at slow speeds (89.6% vs 98.0%): the far-side command gives no extra speed on a torque-limited arm and makes the paddle overshoot past the gate. The 2D gain was a property of the 2D servo model.

Figure 10. Band force: the closed hysteresis loop

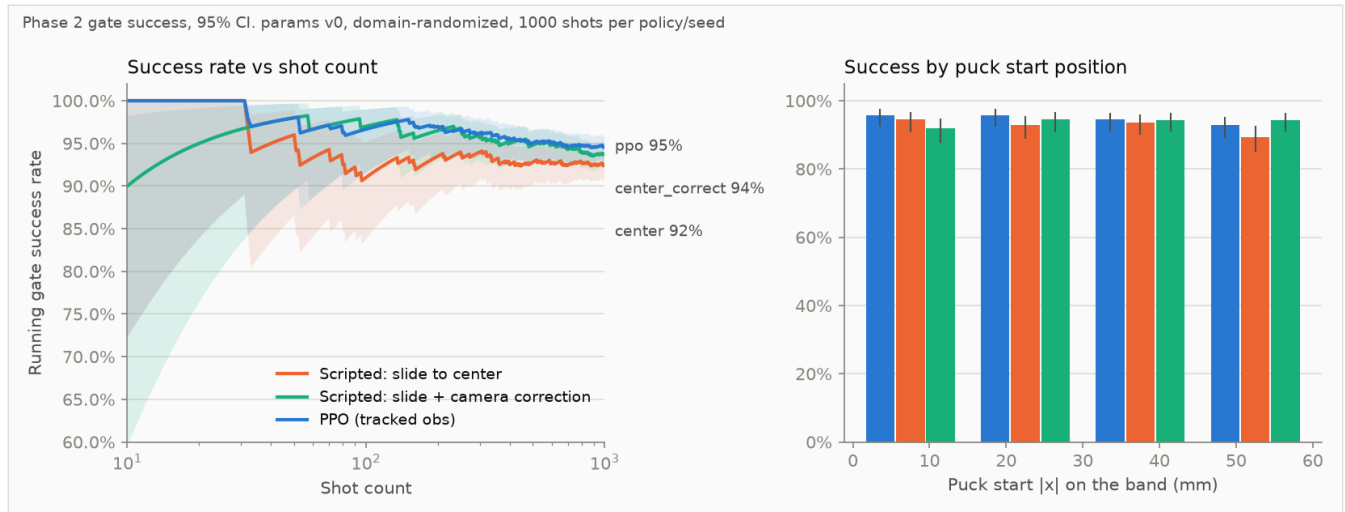


What it shows. Band force on the puck vs the center pull, for the synthetic 'true' band: loading (pull back, blue) and unloading (release, orange) model curves, with the synthetic force-gauge readings (dots, triangles).

How to read it. The area between the curves is the energy lost per cycle. Both curves start and end at the same points.

What we see. The loop closes at 0 and at the maximum pull, as the plan required. The force reaches ~8 N at 30 mm. The fit recovers this band from the gauge readings (stiffness 236 vs 230 N/m, hysteresis 0.220 vs 0.22).

Figure 11. Phase 2 evaluation: gate success

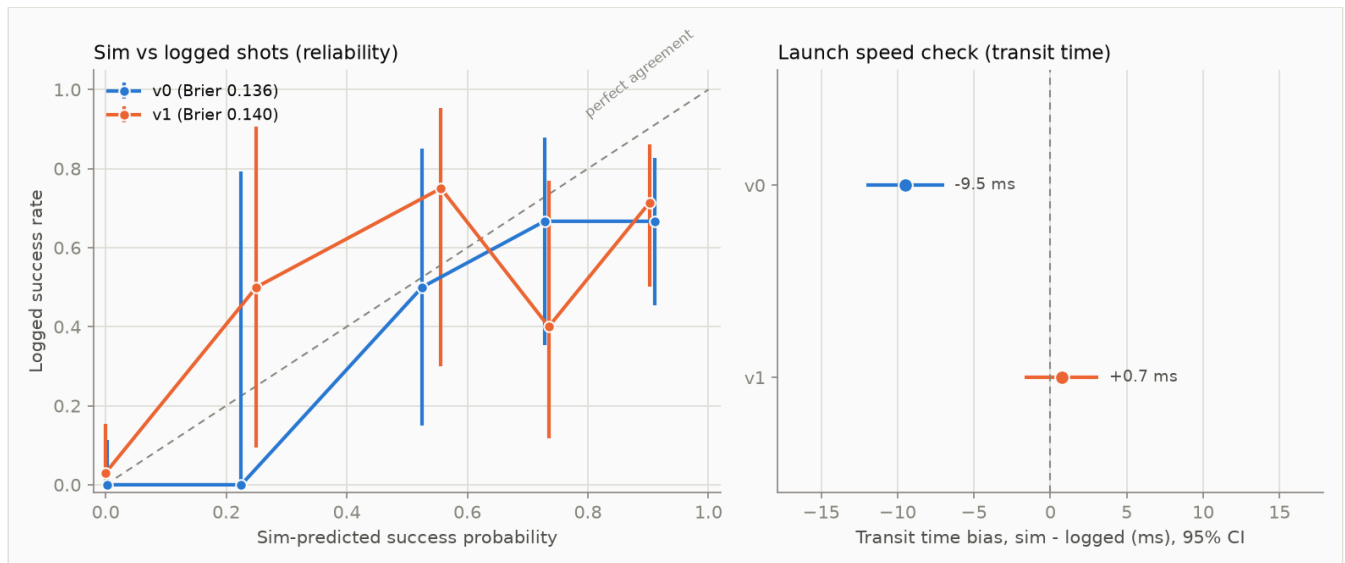


What it shows. Left: running success rate vs shot count (log scale, from 10 shots) with 95% Wilson bands. Right: success by the puck's start position on the band ($|x|$ bins).

How to read it. The running rate shows how many shots are needed for a stable estimate. All policies use the same shots.

What we see. PPO 94.6%, slide + camera correction 93.8%, slide to center 92.5%. The rates settle after ~200 shots. Success does not depend on the start position, because the slide removes the start offset. The remaining misses come from the release scatter and the placement noise (placeholders).

Figure 12. Sysid parity on synthetic logs

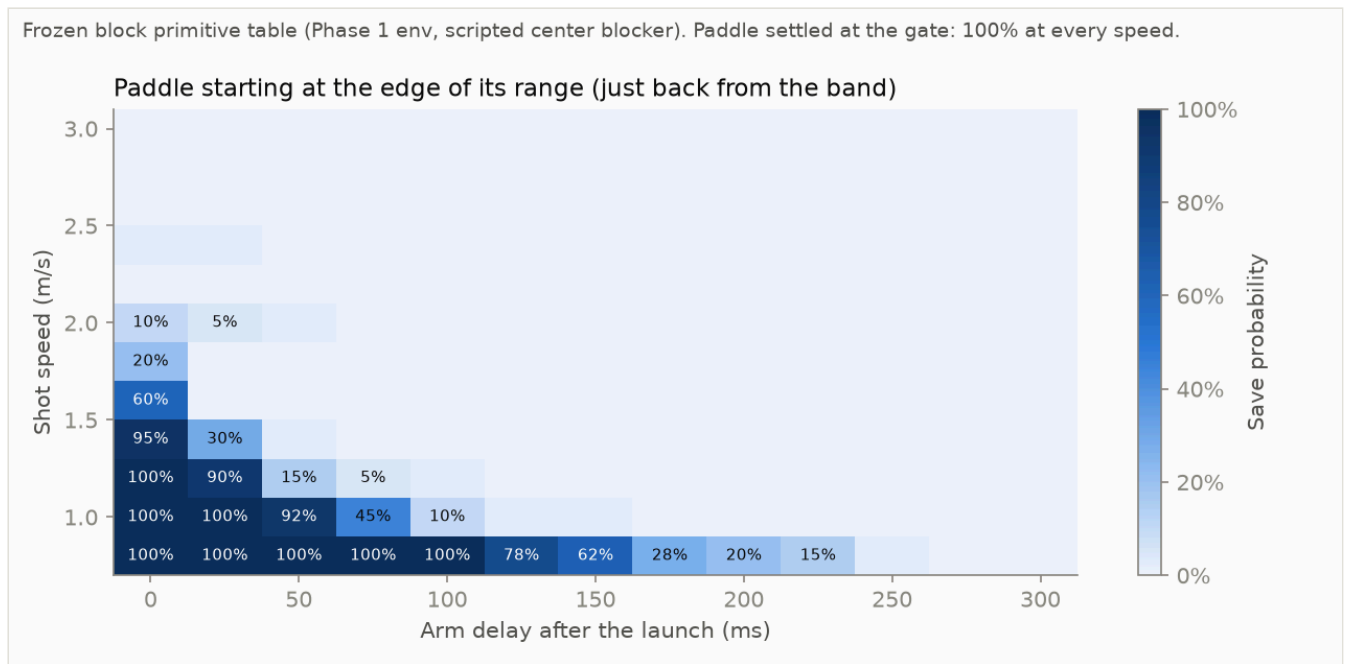


What it shows. Left: reliability diagram (sim-predicted success probability vs logged success) for v0 and the fitted v1. Right: transit-time bias (sim minus logged) with 95% CI.

How to read it. On the left, points on the dashed line mean perfect agreement. On the right, a CI that contains 0 means no detectable speed error.

What we see. The success comparison cannot separate v0 from v1 with 65 shots (Brier 0.136 vs 0.140): the outcome depends on the lateral aim, not on the speed. The transit time can: v0 is 9.5 ms too fast, the fitted v1 agrees (+0.7 ms, CI contains 0).

Figure 13. Frozen block primitive table (Phase 3 input)

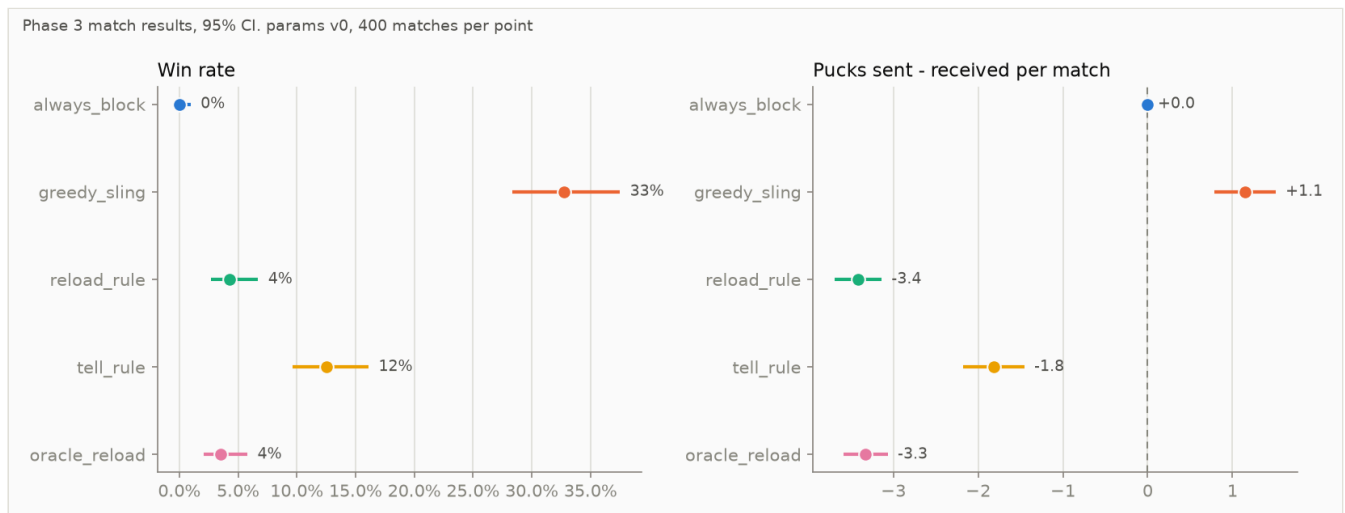


What it shows. Measured save probability for a paddle that starts at the edge of its range (just back from the band), by shot speed and by how late the arm is free after the launch. Darker = more saves; blank cells are below 5%.

How to read it. A paddle already settled at the gate saves 100% at every speed (not shown in the grid).

What we see. The vulnerability window in numbers: from the edge, only slow shots can be stopped (1.0 m/s: 100% with no delay, below 50% after ~75 ms), and almost nothing above 2 m/s. So the gate is safe only if the arm is settled before a fast threat launches.

Figure 14. Phase 3 evaluation: scripted selectors

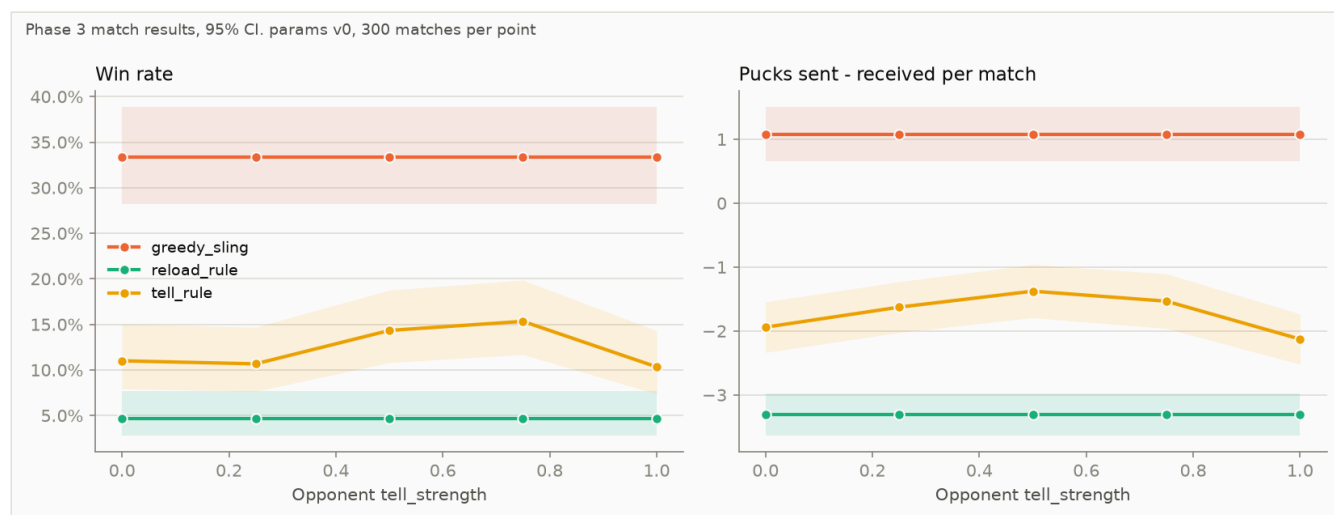


What it shows. Win rate (left) and mean puck difference sent - received per match (right) with 95% CIs, 400 matches each.

How to read it. Each policy keeps a fixed color in every Phase 3 chart.

What we see. Greedy slinging wins most (33%). The plan's reload rule loses 67% of matches, even with the true opponent phase (oracle). The tell rule is between them. Always blocking never wins or loses (0 pucks move).

Figure 15. Phase 3: tell_strength sweep



What it shows. Win rate and puck difference vs the opponent's tell_strength for three policies (300 matches per point).

How to read it. Only the tell rule uses the hand x, so only its line can change with the tell.

What we see. The tell rule is always better than the reload rule, but its gain does not grow clearly with the tell: a sling keeps the arm away for ~3.1 s (about 2.5 opponent shots), and the tell only says something about the current shot.

7. Evaluation results (tables)

All numbers come from the result files in `results/` (95% confidence intervals in brackets). PPO intervals combine the spread across training seeds and the shot sampling; scripted policies use Wilson intervals. All results use placeholder physics values (v0).

Phase 1 save rate vs speed, 2D physics

Speed (m/s)	PPO (3 seeds)	Go to center	Never move
0.80–1.17	100.0% [98.7%, 100.0%]	100.0% [98.7%, 100.0%]	38.3% [33.0%, 43.9%]
1.17–1.53	100.0% [98.7%, 100.0%]	99.7% [98.1%, 99.9%]	39.0% [33.7%, 44.6%]
1.53–1.90	96.3% [93.6%, 97.9%]	90.7% [86.8%, 93.5%]	38.7% [33.3%, 44.3%]
1.90–2.27	85.0% [80.5%, 88.6%]	78.0% [73.0%, 82.3%]	42.3% [36.9%, 48.0%]
2.27–2.63	65.3% [59.8%, 70.5%]	57.3% [51.7%, 62.8%]	36.0% [30.8%, 41.6%]
2.63–3.00	60.3% [54.7%, 65.7%]	57.3% [51.7%, 62.8%]	43.3% [37.8%, 49.0%]

Phase 1 save rate vs speed, MuJoCo physics (sim-to-sim)

Speed (m/s)	PPO (3 seeds)	Go to center	Never move
0.80–1.17	89.6% [83.6%, 95.5%]	98.0% [95.7%, 99.1%]	36.3% [31.1%, 41.9%]
1.17–1.53	75.2% [70.2%, 79.9%]	77.3% [72.3%, 81.7%]	36.3% [31.1%, 41.9%]
1.53–1.90	54.6% [49.0%, 60.2%]	54.7% [49.0%, 60.2%]	33.7% [28.6%, 39.2%]
1.90–2.27	48.7% [43.1%, 54.3%]	48.7% [43.1%, 54.3%]	35.7% [30.5%, 41.2%]
2.27–2.63	36.0% [30.8%, 41.6%]	36.0% [30.8%, 41.6%]	30.3% [25.4%, 35.8%]
2.63–3.00	38.7% [33.3%, 44.3%]	38.7% [33.3%, 44.3%]	34.7% [29.5%, 40.2%]

Phase 1 time-to-cover grid ('go to center'): 2D / MuJoCo

Save rate by shot speed (rows) and paddle start offset as a fraction of the pan range (columns). This is the table Phase 3 needs; note how much lower MuJoCo is for far starts.

Speed (m/s)	0.00–0.25	0.25–0.50	0.50–0.75	0.75–1.00
0.80–1.17	100.0% / 100.0%	100.0% / 100.0%	100.0% / 97.7%	100.0% / 94.5%
1.17–1.53	100.0% / 100.0%	100.0% / 96.1%	100.0% / 78.7%	98.6% / 31.0%
1.53–1.90	100.0% / 100.0%	100.0% / 80.8%	98.6% / 34.7%	64.0% / 0.0%
1.90–2.27	100.0% / 100.0%	100.0% / 72.0%	85.7% / 8.6%	18.8% / 0.0%
2.27–2.63	100.0% / 97.1%	96.2% / 51.3%	38.2% / 0.0%	1.2% / 0.0%
2.63–3.00	100.0% / 92.0%	94.3% / 50.0%	23.9% / 0.0%	1.4% / 0.0%

Phase 2 gate success (1000 shots per policy and seed)

Policy	All starts	x0 0–15 mm	x0 15–30 mm	x0 30–45 mm	x0 45–60 mm
PPO (3 seeds)	94.6% [93.0%, 95.8%]	95.7% [92.3%, 97.7%]	95.6% [92.4%, 97.7%]	94.5% [90.9%, 96.5%]	92.8% [89.1%, 95.4%]
Slide + camera correction	93.8% [92.1%, 95.1%]	91.9% [87.7%, 94.7%]	94.5% [90.8%, 96.8%]	94.3% [90.9%, 96.5%]	94.3% [90.9%, 96.5%]
Slide to center	92.5% [90.7%, 94.0%]	94.4% [90.7%, 96.7%]	92.8% [88.8%, 95.5%]	93.6% [90.0%, 96.0%]	89.4% [85.2%, 92.6%]

Sysid parity on synthetic logs (65 shots, 40 sim replays each)

Params	Logged success	Sim success	Brier	Transit bias (sim – logged)	Transit RMSE
v0	33.8%	43.2%	0.136	-9.5 ms [-12.0, -7.0]	13.7 ms
v1	33.8%	39.0%	0.140	+0.7 ms [-1.7, +3.2]	9.5 ms

Sysid recovery on synthetic logs (seed 0)

Parameter	v0	Truth	Fit ± standard error
Band stiffness (N/m)	200	230	236 ± 6
Band exponent	1.0	1.2	1.18 ± 0.02
Hysteresis loss	0.15	0.22	0.220 ± 0.002
Servo latency (s)	0.030	0.042	0.042
Servo lag τ (s)	0.050	0.065	0.065
Servo max rate (deg/s)	300	250	255 (2% high, encoder quantization)
Servo deadband (deg)	0.5	0.4	0.40
Compliance (m/N)	0.0010	0.0014	0.00142 ± 0.00003
Friction μ	0.20	0.24	0.240 ± 0.005
Wall restitution	0.85	0.78	0.80 ± 0.02
Energy transfer	0.70	0.62	0.614 ± 0.009

Over 6 synthetic data sets, all shot-fitted values were within about one standard error of the truth.

Phase 3 scripted selectors (400 matches each, tell_strength 0.8)

Policy	Win	Loss	Time-out	Sent – received	Slings / match	Conceded while away / match	Save rate
always_block	0.0% [0.0%, 1.0%]	0.0%	100.0%	+0.00 [+0.00, +0.00]	0.0	0.0	100.0%
greedy_sling	32.8% [28.3%, 37.5%]	16.5%	50.7%	+1.15 [+0.79, +1.51]	23.0	20.0	0.0%
reload_rule	4.2% [2.7%, 6.7%]	67.2%	28.5%	-3.42 [-3.69, -3.14]	15.2	17.0	2.6%
tell_rule	12.5% [9.6%, 16.1%]	46.5%	41.0%	-1.81 [-2.17, -1.46]	18.8	18.9	0.3%
oracle_reload	3.5% [2.1%, 5.8%]	62.7%	33.8%	-3.33 [-3.59, -3.07]	15.5	16.9	2.9%

Phase 3 tell_strength sweep (300 matches per cell): win rate / puck difference

Policy	tell 0.00	tell 0.25	tell 0.50	tell 0.75	tell 1.00
greedy_sling	33.3% / +1.1	33.3% / +1.1	33.3% / +1.1	33.3% / +1.1	33.3% / +1.1
reload_rule	4.7% / -3.3	4.7% / -3.3	4.7% / -3.3	4.7% / -3.3	4.7% / -3.3
tell_rule	11.0% / -1.9	10.7% / -1.6	14.3% / -1.4	15.3% / -1.5	10.3% / -2.1

Phase 3 regime maps (scripted policies, quick runs)

Opponent speed and accuracy (150 matches per cell; win rate W, loss rate L):

Opponent reload, threat probability	Greedy sling	Reload rule	Tell rule	Oracle reload
0.3 s, 0.5	W 0.06 / L 0.69	W 0.00 / L 0.97	W 0.00 / L 0.90	W 0.00 / L 0.99
0.6 s, 0.5 (default)	W 0.33 / L 0.11	W 0.03 / L 0.59	W 0.17 / L 0.40	W 0.01 / L 0.63
1.0 s, 0.5	W 0.90 / L 0.01	W 0.22 / L 0.13	W 0.63 / L 0.03	W 0.24 / L 0.14
0.6 s, 0.8	W 0.00 / L 0.99	W 0.00 / L 1.00	W 0.00 / L 1.00	W 0.00 / L 1.00
1.0 s, 0.8	W 0.01 / L 0.57	W 0.00 / L 1.00	W 0.00 / L 0.96	W 0.00 / L 0.98

Robot speed (move, fetch and sling times scaled by f; 300 matches per cell; win rate, save rate):

f (sling-and-return cycle)	Greedy	Reload rule	Tell rule, tell 0	Tell rule, tell 1
1.0 (3.1 s)	33%, 0%	5%, 2%	11%, 0%	10%, 1%
0.5 (1.6 s)	100%, 0%	100%, 5%	100%, 2%	100%, 3%
0.3 (0.9 s)	100%, 0%	100%, 30%	100%, 7%	100%, 14%

Under the placeholder rule "first empty side wins", the match is a throughput race: when the robot is faster than the opponent's threat rate, every strategy wins; when it is slower, defending does not pay.

8. Code reference: every file, class and function

For each file: what it is for, how it works, important notes, and a table of its classes, functions and methods (read from the code). Small helper functions nested inside other functions are part of their parent's description.

Core

src/slingpuck/config.py

Configuration loading, parameter versions, randomization · 211 lines

Every other module reads its numbers from one merged configuration dictionary built here. `load_config()` merges three YAML files from `configs/`: `servo.yaml`, then `physics_params_v<N>.yaml`, then `env.yaml`. A later file overrides an earlier one key by key.

Physics parameters are versioned. `sysid` writes `physics_params_v1.yaml`, `v2`, ...; 'latest' loads the highest version, and the loader checks that the version inside the file matches its name.

Every value that is not measured is listed under 'placeholders:' in its file. `strict=True` refuses to load while any placeholder or null value remains, so no result can be reported against real data by accident with guessed numbers.

`cfg['meta']` records the params version, the placeholder list, the git hash and whether the working tree had uncommitted changes. `save_run_config()` writes all of this, plus the seed, next to every training run.

Important

- `sample_randomized()` is the domain randomization: it returns a changed copy and never changes the input, and it records the sampled values in `meta['randomized']`.

Kind	Name	Arguments	What it does
class	<code>ConfigError</code>	<code>class(ValueError)</code>	Error raised for a bad or uncalibrated config.
function	<code>config_dir</code>	<code>(path=None)</code>	Return the config folder: the argument, else <code>\$SLINGPUCK_CONFIG_DIR</code> , else <code>configs/</code> .
function	<code>available_param_versions</code>	<code>(cfg_dir=None)</code>	List the version numbers of the <code>physics_params_v<N>.yaml</code> files in a folder.
function	<code>resolve_params_file</code>	<code>(version='latest', cfg_dir=None)</code>	Map "latest", "v3", "3" or 3 to <code>configs/physics_params_v3.yaml</code> .
function	<code>_read_yaml</code>	<code>(path)</code>	Read one YAML file (empty file -> empty dict).
function	<code>deep_merge</code>	<code>(base, override)</code>	Merge two nested dicts; values from 'override' win, key by key.
function	<code>get_path</code>	<code>(cfg, dotted)</code>	Read a value by dotted path, e.g. 'puck.mass_kg'.
function	<code>set_path</code>	<code>(cfg, dotted, value)</code>	Write a value by dotted path.
function	<code>_find_nulls</code>	<code>(node, prefix='')</code>	List the dotted paths of all null values.
function	<code>git_state</code>	<code>()</code>	Return the git hash and whether the working tree has uncommitted changes.
function	<code>load_config</code>	<code>(params_version='latest', strict=False, cfg_dir=None, overrides=None)</code>	Load and merge configs.
function	<code>sample_randomized</code>	<code>(cfg, rng)</code>	Return a copy of <code>cfg</code> with domain-randomized physics values.
function	<code>save_run_config</code>	<code>(cfg, run_dir, seed=None)</code>	Write the full merged config, params version, git state and seed to <code>run_dir</code> .

src/slingpuck/kinematics.py

Goalkeeper kinematics shared by the sim and the robot · 69 lines

GoalkeeperGeometry is the only mapping from a policy action in $[-1, 1]$ to a board-frame pan angle and then to the SO-101 shoulder_pan joint command. The env and the LeRobot interface both use it, so the sim and the robot cannot disagree (the original code mapped action 1.0 to 0.19 rad in the sim and to 0.5 rad on the robot).

It also computes the paddle pose on its arc (center, angle, velocity, angular velocity) for the physics sim. pan = 0 puts the paddle center on the gate line; positive pan moves it toward +x.

Important

- `from_config()` checks that the base is behind the paddle line and that the paddle stays inside the board at full pan; it raises an error otherwise.

Kind	Name	Arguments	What it does
class	GoalkeeperGeometry	class	Base position, arm radius, pan range, paddle size and joint zero offset.
method	GoalkeeperGeometry.from_config	classmethod (cfg)	Build from the config and check that the geometry is valid.
method	GoalkeeperGeometry.action_to_pan	(action)	Policy action in $[-1, 1]$ to board-frame pan angle.
method	GoalkeeperGeometry.pan_to_action	(pan)	Inverse of action_to_pan (clipped to $[-1, 1]$).
method	GoalkeeperGeometry.pan_to_joint	(pan)	Board-frame pan angle to the SO-101 shoulder_pan joint command.
method	GoalkeeperGeometry.joint_to_pan	(joint)	Joint reading to board-frame pan angle.
method	GoalkeeperGeometry.paddle_center	(pan)	Paddle center position on its arc for a pan angle.
method	GoalkeeperGeometry.paddle_state	(pan, pan_vel, restitution)	PaddleState (pose, velocity, angular velocity) for the physics sim.

Physics

src/slingpuck/physics/backend.py

Physics backend interface · 62 lines

Defines the PuckPhysicsBackend abstract class (reset, get_state, set_paddle, step, kinetic_energy) and the small data classes it uses: PaddleState (a kinematic box), GateCrossing and StepEvents.

The envs talk only to this interface, so the fast 2D sim and the MuJoCo sim can replace each other without env changes (the plan asked for a swappable backend).

Board frame: origin at the board center, x across the width, y along the length; the agent half is $y < 0$, the opponent half $y > 0$, and the divider with the gate is at $y = 0$.

Kind	Name	Arguments	What it does
class	GateCrossing	class	A puck crossed $y = 0$: puck index, direction (+1 to the opponent), x.
class	PaddleState	class	Kinematic box moved by the arm.
class	StepEvents	class	Events of one step: gate crossings and paddle contacts.
class	PuckPhysicsBackend	class(ABC)	Abstract interface every puck physics backend implements.
method	PuckPhysicsBackend.reset	(pos, vel)	Set puck positions and velocities, shape (n_pucks, 2).
method	PuckPhysicsBackend.get_state	()	Return copies of (pos, vel), shape (n_pucks, 2).
method	PuckPhysicsBackend.set_paddle	(paddle)	Set the paddle pose for the next step(s).
method	PuckPhysicsBackend.step	(dt)	Advance the sim by dt and return the events in this step.
method	PuckPhysicsBackend.kinetic_energy	()	Total translational kinetic energy of all pucks, in J.

src/slingpuck/physics/puck_dynamics.py

Fast2DPuckSim: the fast planar puck sim · 241 lines

Planar pucks with Coulomb sliding friction (constant deceleration $\mu \cdot g$, stop without reversal), side and end walls, the center divider with the gate gap, puck-puck contacts, and an optional kinematic paddle (an oriented box with linear and angular velocity).

`step(dt)` returns `StepEvents`: the gate crossings (with direction) and the paddle contacts.

How it works

- Contacts reflect the normal velocity relative to the surface, scaled by the restitution, and keep the tangential velocity. Without a moving paddle the kinetic energy can never increase (tested).
- The per-puck contact code uses Python floats, not NumPy, because NumPy call overhead is larger than the arithmetic for 1-2 values. It is 5 times faster than the first NumPy version and gives the same events (max state difference $7e-13$ over 200 random trials).

Kind	Name	Arguments	What it does
class	<code>BoardGeometry</code>	class	Inner board size, gate width, divider thickness, wall restitution.
method	<code>BoardGeometry.from_config</code>	classmethod (cfg)	Build from <code>cfg['board']</code> .
method	<code>BoardGeometry.half_length</code>	property ()	Half of the inner length.
method	<code>BoardGeometry.half_width</code>	property ()	Half of the inner width.
method	<code>BoardGeometry.divider_boxes</code>	()	Two divider blocks beside the gate, rows of [xmin, xmax, ymin, ymax].
class	<code>PuckParams</code>	class	Puck radius, mass, friction and puck-puck restitution.
method	<code>PuckParams.from_config</code>	classmethod (cfg)	Build from <code>cfg['puck']</code> .
function	<code>_reflect</code>	(vx, vy, nx, ny, e, sx=0.0, sy=0.0)	Reflect the velocity component along normal n (relative to surface velocity s) if approaching.
class	<code>Fast2DPuckSim</code>	class(PuckPhysicsBackend)	The fast planar puck simulator (implements PuckPhysicsBackend).
method	<code>Fast2DPuckSim.__init__</code>	(board, puck, n_pucks=1)	Check restitutions and that the gate is wider than the puck.
method	<code>Fast2DPuckSim.from_config</code>	classmethod (cfg, n_pucks=1)	Build from the merged config.
method	<code>Fast2DPuckSim.reset</code>	(pos, vel)	Set positions and velocities; reject pucks outside the board.
method	<code>Fast2DPuckSim.get_state</code>	()	Copies of positions and velocities.
method	<code>Fast2DPuckSim.set_paddle</code>	(paddle)	Set (or remove) the kinematic paddle for the next steps.
method	<code>Fast2DPuckSim.kinetic_energy</code>	()	Total translational kinetic energy.
method	<code>Fast2DPuckSim.step</code>	(dt)	Advance dt: friction, motion, paddle, walls, divider, puck-puck; return events.
method	<code>Fast2DPuckSim._step_puck</code>	(p, v, dt, decel, side_before)	Advance one puck in place (p, v are [x, y] lists).
method	<code>Fast2DPuckSim._collide_paddle</code>	(x, y, vx, vy, r)	Circle vs oriented box contact, using the paddle surface velocity.
method	<code>Fast2DPuckSim._collide_pucks</code>	()	Equal-mass impulse between overlapping pucks.

src/slingpuck/physics/servo_model.py

Servo model (Feetech STS3215) · 54 lines

Each step: command latency (exactly $\text{round}(\text{latency}/dt)$ steps) -> deadband (errors smaller than the deadband are ignored) -> first-order lag (exact discretization of $\tau \cdot dx/dt = \text{target} - x$) -> rate limit (default 300 deg/s). Works for one joint or an array of joints.

Important

- The MuJoCo sim-to-sim test showed that this first-order model is much faster at the start of a move than the torque-limited MuJoCo arm. The real step response must decide which is right.

Kind	Name	Arguments	What it does
function	<code>latency_steps</code>	(<code>latency_s</code> , <code>dt</code>)	Number of whole steps of delay.
class	<code>ServoModel</code>	class	Position servo: latency, deadband, first-order lag, rate limit.
method	<code>ServoModel.__init__</code>	(<code>config</code> , <code>dt</code> , <code>init_pos=0.0</code>)	Read the servo config (deg -> rad) and set up the latency queue.
method	<code>ServoModel.effective_latency_s</code>	property ()	Latency after rounding to whole steps.
method	<code>ServoModel.reset</code>	(<code>pos=0.0</code>)	Set the position and fill the latency queue with it.
method	<code>ServoModel.step</code>	(<code>target</code>)	Apply one command and return the new position.

src/slingpuck/physics/band_model.py

Sling band: pre-tension, nonlinear tension, closed hysteresis loop, release · 147 lines

The band runs between two pegs at (+span/2, band_y). The arm pulls the puck back to point P; the band makes a V and pushes P forward with force $T*(u_{left} + u_{right})$.

The band is pre-stretched (natural length < peg span). The pre-tension gives most of the launch energy: without it, a 3 cm pull would launch a 27 g puck at about 0.4 m/s, too slow to reach the gate.

How it works

- Loading tension $T_l(s) = k*s/(s/1\text{ cm})^{(exponent-1)}$, s = band length - natural length.
- Unloading $T_u(s) = T_l(s) - h*4x(1-x)$, $x = (s-s_0)/(s_{max}-s_0)$. The bump is zero at both ends, so the loop closes, and h is chosen so the cycle loses exactly `hysteresis_loss_factor` of the loading work.
- `release()` integrates the puck from rest under the unloading force and friction until it crosses the band line, then scales the speed by $\sqrt{\text{energy_transfer}}$. A pull too small to beat friction leaves the puck behind the band (`launched = False`).

Important

- **Main finding:** the pull is small compared with the band span, so $u_{left} + u_{right}$ points almost straight ahead for any pull angle. A 17 deg pull steers the puck by less than 1 deg. This is why Phase 2 aims by sliding the puck along the band (placement).

Kind	Name	Arguments	What it does
class	<code>BandParams</code>	class	Peg span, natural length, stiffness, exponent, hysteresis, energy transfer.
method	<code>BandParams.from_config</code>	classmethod (<code>cfg</code>)	Build from <code>cfg['band']</code> .
class	<code>ReleaseResult</code>	class	Exit position and velocity, release time, band work, launched flag.
class	<code>BandModel</code>	class	Band geometry, tension laws and release dynamics.
method	<code>BandModel.__init__</code>	(<code>params</code> , <code>band_y</code>)	Check the parameters and place the anchors on the band line.
method	<code>BandModel.from_config</code>	classmethod (<code>cfg</code>)	Build from the config (band line from the end-wall offset).
method	<code>BandModel.tension_load</code>	(<code>s</code>)	Loading tension for an elongation.
method	<code>BandModel.work_load</code>	(<code>s0</code> , <code>s1</code>)	Integral of T_l from s_0 to s_1 .
method	<code>BandModel.tension_unload</code>	(<code>s</code> , <code>s_max</code>)	Unloading tension on the way back from s_{max} to the rest elongation.
method	<code>BandModel.elongation</code>	(<code>pos</code>)	Band length through a point minus the natural length.
method	<code>BandModel.direction_sum</code>	(<code>pos</code>)	$u_{left} + u_{right}$: the force direction times sin-like factor (zero on the band line).
method	<code>BandModel.force_load</code>	(<code>pos</code>)	Band force on the puck at <code>pos</code> while the band is being stretched.

Kind	Name	Arguments	What it does
method	<code>BandModel.release</code>	<code>(pull_pos, mass, mu, dt=0.0001, max_t=0.3)</code>	Integrate the release from a pulled point; return a <code>ReleaseResult</code> .

src/slingpuck/physics/arm_deflection.py

Arm deflection under band load · 53 lines

The arm gives way a little under the band force: actual pull = commanded pull - compliance * F_{par} , where F_{par} is the band force component that resists the pull. `solve_pull()` finds the actual pull by bisection, which always converges because $d + c * F_{\text{par}}(d)$ increases with d .

`pull_direction(angle)` gives the unit pull vector (angle 0 = straight back, away from the gate).

Important

- Only deflection along the pull direction is modeled. A bug (compliance 0 returned half the pull) was found by a test and fixed.

Kind	Name	Arguments	What it does
class	<code>PullResult</code>	class	Actual puck position, actual pull distance, band force there.
function	<code>pull_direction</code>	<code>(angle_rad)</code>	Unit pull direction.
function	<code>solve_pull</code>	<code>(band, start, angle_rad, pull_cmd_m, compliance_m_per_n, tol=1e-09)</code>	Solve actual pull = command - compliance * resisting force (bisection).

src/slingpuck/physics/mujoco_goalkeeper.py

MuJoCo physics for the goalkeeper (board, puck, SO-101 arm) · 499 lines

Builds a MuJoCo scene in code (MjSpec) from the merged config: the real-size board, walls, divider and gate, the puck, and the RobotStudio SO-101 model (STS3215 actuators, kp 998, +-2.94 N m) holding a paddle in front of the gate. It implements `PuckPhysicsBackend`, so the Phase 1 env runs on it.

How it works

- Arm placement: the base is turned 90 deg so the arm reaches toward the gate, and placed so that the pan axis is exactly at `robot.base_xy_m`.
- Arm pose: least-squares inverse kinematics puts the gripper fingertips at the paddle center; the paddle is a box fixed to the gripper (a custom mount) and absorbs a 1 mm offset of the pitch-joint plane. The hold targets are corrected for gravity sag. The paddle is on the 2D arc within 0.2 mm.
- Puck: planar (x, y, spin) with an applied Coulomb friction force. A free puck tumbled off the board, and MuJoCo contact friction made it hop and lose friction after wall hits.
- Contacts: only explicit pairs collide, frictionless like the 2D sim. They use direct stiffness/damping; the damping is calibrated in the real scene (24 damping values x 5 impact phases) so the measured restitution matches the config. Paddle restitution is therefore the effective value on the compliant arm.
- Timestep 0.5 ms; about 180 episodes per second.

Kind	Name	Arguments	What it does
function	<code>_rotz</code>	<code>(yaw)</code>	Rotation matrix about z.
function	<code>_pan_anchor_local</code>	<code>()</code>	shoulder_pan axis position in the SO-101 base frame.
class	<code>SceneLayout</code>	class	Board-frame positions derived from the config.
method	<code>SceneLayout.from_config</code>	classmethod (cfg)	Compute base pose and paddle target from the config.
class	<code>ArmHoldPose</code>	class	IK joints, sag-corrected hold targets, settled pose, paddle mount pose, errors.
function	<code>_divider_boxes</code>	<code>(cfg)</code>	The two divider blocks beside the gate.

Kind	Name	Arguments	What it does
function	<code>_build_spec</code>	(<code>cfg</code> , <code>layout</code> , <code>pose</code>)	Build the MjSpec: SO-101, board, walls, divider, puck, paddle, pairs, lights, cameras.
function	<code>_stiffness</code>	(<code>cfg</code>)	Contact stiffness for the configured contact time.
function	<code>_add_pairs</code>	(<code>spec</code> , <code>cfg</code>)	Contact pairs.
class	<code>ContactCalibration</code>	<code>class</code>	Measured restitution vs contact damping, for walls and for the arm-held paddle.
method	<code>ContactCalibration.solref</code>	(<code>kind</code> , <code>restitution</code>)	Interpolate (-stiffness, -damping) for a target restitution.
function	<code>_qadr</code>	(<code>model</code> , <code>names</code>)	qpos addresses of joints by name.
function	<code>_park_puck</code>	(<code>model</code> , <code>data</code> , <code>cfg</code> , <code>xy</code> =(0.0, 0.1))	Put the puck at rest (used while the arm settles).
function	<code>solve_hold_pose</code>	(<code>cfg</code> , <code>layout</code>)	Inverse kinematics for the paddle pose at <code>pan = 0</code> , then gravity-sag compensation.
function	<code>calibrate_contacts</code>	(<code>sim</code> , <code>impact_speed</code> =2.0, <code>n</code> =24, <code>n_phases</code> =5)	Measure restitution vs damping in the real scene (no friction during the test).
function	<code>_pose_key</code>	(<code>cfg</code>)	Cache key of the config values that change the arm pose.
class	<code>MujocoGoalkeeperSim</code>	<code>class(PuckPhysicsBackend)</code>	PuckPhysicsBackend on MuJoCo.
method	<code>MujocoGoalkeeperSim.__init__</code>	(<code>cfg</code>)	Solve (or reuse) the arm pose, compile the model, calibrate contacts.
method	<code>MujocoGoalkeeperSim.apply_params</code>	(<code>cfg</code>)	Update friction, restitution, puck mass and gate width in place (no recompile).
method	<code>MujocoGoalkeeperSim.reset_arm</code>	(<code>pan</code>)	Put the arm at a <code>pan</code> angle in the settled hold pose.
method	<code>MujocoGoalkeeperSim.set_pan_target</code>	(<code>pan</code>)	Set the <code>shoulder_pan</code> actuator target.
method	<code>MujocoGoalkeeperSim.pan</code>	property ()	Current board-frame <code>pan</code> angle.
method	<code>MujocoGoalkeeperSim.pan_vel</code>	property ()	Current <code>pan</code> velocity.
method	<code>MujocoGoalkeeperSim.paddle_center</code>	()	Paddle center in the world.
method	<code>MujocoGoalkeeperSim.reset</code>	(<code>pos</code> , <code>vel</code>)	Set the puck position and velocity.
method	<code>MujocoGoalkeeperSim.pos</code>	property ()	Puck position (1, 2).
method	<code>MujocoGoalkeeperSim.vel</code>	property ()	Puck velocity (zero below 1 mm/s, so 'stopped' works).
method	<code>MujocoGoalkeeperSim.get_state</code>	()	Position and velocity.
method	<code>MujocoGoalkeeperSim.set_paddle</code>	(<code>paddle</code>)	No-op: the arm moves the paddle.
method	<code>MujocoGoalkeeperSim.kinetic_energy</code>	()	Puck kinetic energy.
method	<code>MujocoGoalkeeperSim.step</code>	(<code>dt</code>)	Apply friction, step MuJoCo, detect paddle contacts and gate crossings.
class	<code>MujocoPanServo</code>	<code>class</code>	Servo adapter: command latency in software, then the MuJoCo STS3215 actuator.
method	<code>MujocoPanServo.__init__</code>	(<code>sim</code> , <code>servo_cfg</code> , <code>dt</code>)	Set up the command latency queue.
method	<code>MujocoPanServo.reset</code>	(<code>pan</code>)	Reset the arm and fill the queue.
method	<code>MujocoPanServo.step</code>	(<code>target_pan</code>)	Delay the command, send it to the actuator, return the <code>pan</code> .
method	<code>MujocoPanServo.pos</code>	property ()	<code>Pan</code> angle (same interface as <code>ServoModel</code>).

Kind	Name	Arguments	What it does
method	<code>MujocoPanServo.vel</code>	property ()	Pan velocity.

Sensing

src/slingpuck/sensing/camera_model.py

Overhead camera model · 61 lines

60 Hz frames with configurable latency, Gaussian position noise and random dropouts, using a seeded random generator. Each Frame carries its capture time and its arrival time. `observe(t, true_xy)` returns only the frames that arrive at this step, so a frame is never delivered twice (the original code fed the same stale frame to the tracker again and again).

Kind	Name	Arguments	What it does
class	<code>Frame</code>	class	Capture time, arrival time and measured position of one frame.
class	<code>CameraModel</code>	class	Frame rate, latency, noise and dropouts.
method	<code>CameraModel.__init__</code>	(config, rng=None)	Read the camera config.
method	<code>CameraModel.reset</code>	(t0=0.0, rng=None)	Restart frame timing and clear pending frames.
method	<code>CameraModel.observe</code>	(t, true_xy)	Capture when a frame is due; return frames that arrive now.
method	<code>CameraModel._capture</code>	(t, true_xy)	Take one frame: dropout or noisy measurement.

src/slingpuck/sensing/kalman_tracker.py

Constant-velocity Kalman tracker with latency compensation · 81 lines

State [x, y, vx, vy], white-noise-acceleration process model, measurement noise from the camera config. It starts from the first frame. `update()` filters each frame at its capture time; `estimate(t)` predicts to the current time, which removes the camera latency (error < 25% of the raw frame in tests).

Important

- Limits: it does not model wall bounces or a sudden launch from rest; 80 ms after a launch the estimate is about 10 cm behind. For a still puck the Phase 2 env uses the plain frame mean instead.

Kind	Name	Arguments	What it does
function	<code>_transition</code>	(dt, accel_std)	State transition and process noise for a time step.
class	<code>KalmanTracker</code>	class	Constant-velocity Kalman filter for one puck.
method	<code>KalmanTracker.__init__</code>	(meas_noise_std_m, accel_noise_std_m_s2, init_vel_std_m_s=2.0)	Set measurement and process noise.
method	<code>KalmanTracker.from_config</code>	classmethod (cfg)	Build from the camera and tracker config.
method	<code>KalmanTracker.reset</code>	()	Forget the state.
method	<code>KalmanTracker.initialized</code>	property ()	True after the first frame.
method	<code>KalmanTracker.update</code>	(frame)	Predict to the frame capture time and update (Joseph form).
method	<code>KalmanTracker.estimate</code>	(t_now)	Return (pos, vel, cov) predicted to t_now, or None before the first frame.

Environments

src/slingpuck/envs/goalkeeper_env.py

Phase 1 env: goalkeeper (recovery task) · 268 lines

The opponent's puck rests at the opponent band line, then launches toward the gate. The policy commands the pan joint; the paddle swings on its arc. The policy sees only the Kalman-tracked puck, the pan encoder and its previous

action (8 values). The true state is in `info['privileged']` (16 values) for an asymmetric critic.

Recovery task (user decision): the gate is only ~1.5 mm wider than the puck, so a settled paddle blocks everything. The paddle is therefore locked at a random offset until the launch (plus `release_delay_s`), as if the arm were busy with a sling. The result is a time-to-cover measurement.

Reward: +1 save, -1 goal, 0 no threat, minus $0.01 \cdot |\text{change of action}|$. With `threats_only`, shots are resampled until they would score with no paddle.

How it works

- Three physics hooks (`_build_physics`, `_reset_physics`, `_physics_step`) are the only place that touches the physics; `MujocoGoalkeeperEnv` replaces just these three.
- Domain randomization is sampled again at every reset. `substep_hook` lets the viewers record frames.

Kind	Name	Arguments	What it does
class	<code>Shot</code>	<code>class</code>	Start, velocity, speed, pre-launch time, threat flag and tries of one shot.
class	<code>GoalkeeperEnv</code>	<code>class(gym.Env)</code>	Phase 1 Gymnasium env.
method	<code>GoalkeeperEnv.__init__</code>	<code>(config)</code>	Read the task config and define the spaces.
method	<code>GoalkeeperEnv.reset</code>	<code>(seed=None, options=None)</code>	options: speed (m/s), start_pan (rad), randomize (bool).
method	<code>GoalkeeperEnv._sample_shot</code>	<code>(options)</code>	Sample start, aim and speed; resample until it is a threat.
method	<code>GoalkeeperEnv._would_score</code>	<code>(start, vel)</code>	Direct shot with no paddle: True if it enters the agent half before it turns back or stops.
method	<code>GoalkeeperEnv.step</code>	<code>(action)</code>	Apply the action for one control period (33 physics steps); return reward and outcome.
method	<code>GoalkeeperEnv._build_physics</code>	<code>(cfg)</code>	Create <code>self.sim</code> (<code>PuckPhysicsBackend</code>) and <code>self.serve</code> (<code>.pos</code> , <code>.vel</code> , <code>.step</code> , <code>.reset</code>).
method	<code>GoalkeeperEnv._reset_physics</code>	<code>(pan0, puck_start)</code>	Physics hook: reset servo, puck and paddle.
method	<code>GoalkeeperEnv._physics_step</code>	<code>(target_pan)</code>	Physics hook: servo step, paddle pose, sim step.
method	<code>GoalkeeperEnv._substep</code>	<code>(target_pan)</code>	Advance one physics step.
method	<code>GoalkeeperEnv._check_outcome</code>	<code>(events)</code>	Decide save / goal / none after a physics step.
method	<code>GoalkeeperEnv._final_outcome</code>	<code>()</code>	Outcome by side when the puck stops or time runs out.
method	<code>GoalkeeperEnv._obs</code>	<code>()</code>	The 8-value policy observation (tracked puck, encoder, previous action).
method	<code>GoalkeeperEnv.privileged_obs</code>	<code>()</code>	The 16-value critic observation (true state, randomized physics).
method	<code>GoalkeeperEnv._info</code>	<code>(outcome)</code>	Info dict: outcome, speed, threat, start pan, release delay, privileged.

`src/slingpuck/envs/mujoco_goalkeeper_env.py`

Phase 1 env on MuJoCo physics · 37 lines

Same task, observation, action, reward and sensing as `GoalkeeperEnv`; only the three physics hooks change. A policy trained on the 2D sim runs here unchanged, which makes this a sim-to-sim test.

Important

- Pan dynamics come from the MuJoCo actuator, not `ServoModel` (only the command latency is shared). The threat check at reset still uses the 2D sim.

Kind	Name	Arguments	What it does
class	<code>MujocoGoalkeeperEnv</code>	<code>class(GoalkeeperEnv)</code>	<code>GoalkeeperEnv</code> with MuJoCo physics hooks.
method	<code>MujocoGoalkeeperEnv.__init__</code>	<code>(config)</code>	Build the MuJoCo sim once.
method	<code>MujocoGoalkeeperEnv._build_physics</code>	<code>(cfg)</code>	Apply randomized parameters in place; set sim and servo.

Kind	Name	Arguments	What it does
method	<code>MujocoGoalkeeperEnv._reset_physics</code>	(pan0, puck_start)	Reset the arm at the start pan and place the puck.
method	<code>MujocoGoalkeeperEnv._physics_step</code>	(target_pan)	Send the pan command and step MuJoCo.

src/slingpuck/envs/sling_env.py

Phase 2 env: targeted shot through the gate · 237 lines

Step 1 (aim): the policy sees the tracked puck at rest and chooses (slide x, pull angle, pull distance). The arm slides the puck along the band (placement noise), pulls it (angle and pull noise) and the arm deflects. Step 2 (correct): the policy sees the pulled puck and the band depth from the camera and applies a small correction; then the band releases and the puck flies on Fast2DPuckSim.

Reward: +1 for crossing the gate, plus $0.3 \cdot \exp(-(\text{miss}/1 \text{ cm})^2)$, where miss is $|x|$ at the divider.

How it works

- `realize_pull()` and `launch_and_fly()` are module-level functions, shared by the env, the scripted planner, the synthetic sysid logs and the parity eval.
- `_observe_still()` uses the mean of 0.3 s of camera frames: for a still puck this gives ~0.7 mm error, while the constant-velocity Kalman filter gave ~2 mm and made the correction harmful.

Kind	Name	Arguments	What it does
class	<code>FlightResult</code>	class	Success, miss distance, exit state and optional trajectory of one shot.
function	<code>slide_limit</code>	(cfg)	Largest $ x $ the puck can be slid to along the band.
function	<code>realize_pull</code>	(cfg, band, x_start, angle, pull, rng, noise_scale=1.0)	Arm pull with execution noise and deflection, kept inside the board.
function	<code>launch_and_fly</code>	(cfg, band, pull_pos, rng, record=False, physics_dt=0.001, full=False)	Release and fly.
class	<code>SlingEnv</code>	class(gym.Env)	Phase 2 Gymnasium env.
method	<code>SlingEnv.__init__</code>	(config)	Read the task config and define the spaces.
method	<code>SlingEnv.scale_aim</code>	(action)	Map the step-1 action to slide x, pull angle and pull distance.
method	<code>SlingEnv.scale_correction</code>	(action)	Map the step-2 action to a lateral, angle and pull correction.
method	<code>SlingEnv.reset</code>	(seed=None, options=None)	options: x0 (puck start x), randomize (bool).
method	<code>SlingEnv.step</code>	(action)	Step 1: slide and pull (then observe). Step 2 (or step 1 without correction): correct, release, fly, reward.
method	<code>SlingEnv._observe_still</code>	(pos)	Camera watches the still puck for <code>observe_window_s</code> ; returns the mean of the frames.
method	<code>SlingEnv._obs</code>	()	The 7-value policy observation.
method	<code>SlingEnv.privileged_obs</code>	()	The 12-value critic observation.
method	<code>SlingEnv._info</code>	(flight)	Info dict: start x, stage, outcome, miss, exit speed, commands, actual pull.

src/slingpuck/envs/match_env.py

Phase 3 env: event-level match against the scripted opponent · 261 lines

The high-level policy picks one of three frozen primitives every 0.1 s: 0 block, 1 sling, 2 hold. Arm states: gate -> to_band -> slinging -> at_band -> to_gate. The gate is undefended from the sling command until the arm is back and settled (the explicit recovery time from the plan).

Nothing is simulated at 1 kHz: opponent shots fly straight with friction (analytic arrival time); a threat is saved with the probability from the measured block table; a sling succeeds with the Phase 2 probability. About 70,000 decisions per second.

Observation (20): tracked incoming puck (after the camera latency), opponent hand, arm state, puck counts, time.

Privileged (11): opponent phase, time to release, next shot, incoming threat.

Reward and win rule come from 'scoring' (placeholder: +1 sent, -1 received, first empty side wins).

Kind	Name	Arguments	What it does
class	<code>MatchEnv</code>	<code>class(gym.Env)</code>	Phase 3 Gymnasium env (event-level match).
method	<code>MatchEnv.__init__</code>	<code>(config, block_table=None)</code>	Load (or build) the block table, set the sling primitive and the spaces.
method	<code>MatchEnv.reset</code>	<code>(seed=None, options=None)</code>	options: <code>tell_strength</code> (override for sweeps).
method	<code>MatchEnv.step</code>	<code>(action)</code>	Apply the action, advance 0.1 s in 10 ms steps, add win/loss reward.
method	<code>MatchEnv.can_sling</code>	<code>()</code>	True if the arm is free (gate or band), a puck is left and none is in flight.
method	<code>MatchEnv._apply_action</code>	<code>(action)</code>	Start a sling or a return to the gate (busy actions are ignored).
method	<code>MatchEnv._advance</code>	<code>()</code>	Advance the arm state machine, own puck, opponent and incoming pucks by one step.
method	<code>MatchEnv._flight</code>	<code>(shot)</code>	Analytic arrival time of an opponent shot at the gate.
method	<code>MatchEnv._resolve</code>	<code>(f)</code>	Resolve an arriving shot: miss, save (block table) or goal.
method	<code>MatchEnv._match_over</code>	<code>()</code>	Apply the win rule (first side with no pucks).
method	<code>MatchEnv._incoming_state</code>	<code>()</code>	Earliest-arriving opponent puck in flight that the camera has seen, true [x, y, vx, vy].
method	<code>MatchEnv._obs</code>	<code>()</code>	The 20-value policy observation.
method	<code>MatchEnv.privileged_obs</code>	<code>()</code>	The 11-value critic observation.
method	<code>MatchEnv._info</code>	<code>(outcome)</code>	Info dict: outcome, counts, arm state, opponent phase, stats at the end.

src/slingpuck/envs/wrappers.py

Privileged observation wrapper · 33 lines

Turns an env's Box observation into a Dict {'policy': ..., 'privileged': ...}, taking the privileged vector from info. Used with the asymmetric actor-critic policy.

Kind	Name	Arguments	What it does
class	<code>PrivilegedObsWrapper</code>	<code>class(gym.ObservationWrapper)</code>	Dict observation {"policy": actor obs, "privileged": critic-only obs}.
method	<code>PrivilegedObsWrapper.__init__</code>	<code>(env)</code>	Define the Dict observation space.
method	<code>PrivilegedObsWrapper.reset</code>	<code>(**kwargs)</code>	Reset and keep the privileged vector from info.
method	<code>PrivilegedObsWrapper.step</code>	<code>(action)</code>	Step and keep the privileged vector from info.
method	<code>PrivilegedObsWrapper.observation</code>	<code>(obs)</code>	Build the Dict observation.

Opponent and primitives

src/slingpuck/opponent/scripted_opponent.py

Scripted human opponent (Phase 3) · 141 lines

Cycle: reload (the hand fetches a puck) -> place (on the band) -> pull (pull ~ shot speed) -> hold (random pause) -> release, about 1.2 s per shot. Each cycle samples its timings, the launch x, the speed and whether the shot is a threat (launch $|x| < 3$ mm).

The hand covers the loaded puck from the camera, so only the hand is observed (position and velocity along straight segments, plus noise).

tell_strength: during place, pull and hold the hand x is $\text{tell} * x_{\text{launch}} + (1 - \text{tell}) * x_{\text{decoy}}$, with the decoy from the same distribution, so 0 = no information and 1 = exact (tested).

Kind	Name	Arguments	What it does
class	Shot	class	Launch time, launch x, speed and threat flag of one opponent shot.
class	_Cycle	class	Phase boundary times, hand waypoints and the shot of one cycle.
class	Opponent	class	The scripted opponent.
method	Opponent.__init__	(cfg, rng)	Read the opponent config and the band line.
method	Opponent._launch_x	(threat)	Sample a launch x for a threat or a miss.
method	Opponent._start_cycle	(t0, start_pos)	Sample one full cycle: timings, shot, tell, hand waypoints.
method	Opponent.reset	(t0, has_puck)	Start at home; begin a cycle if the opponent has a puck.
method	Opponent.advance	(t, has_puck)	Advance to time t.
method	Opponent.phase	(t)	Current phase name.
method	Opponent.hand_true	(t)	True hand [x, y, vx, vy].
method	Opponent.hand_observed	(t)	True hand plus camera noise.
method	Opponent.time_to_release	(t)	Seconds until the next release (privileged).
method	Opponent.next_shot	property ()	The shot of the current cycle (privileged).

src/slingpuck/primitives/block.py

Frozen block primitive: save-probability table · 114 lines

Measures, once, on the Phase 1 env with a frozen block policy: p_gate(speed) for a paddle settled at the gate when the shot launches, and p_edge(speed, delay) for a paddle that is just back from the band at the edge of its range and free 'delay' seconds after the launch. Cached in cache/ under a hash of everything that changes it (about 40 s to build).

The table shows the vulnerability window: from the edge the paddle saves 100% of 1 m/s shots, 60% at 1.6 m/s and almost none above 2 m/s; a settled paddle saves 100% at every speed.

Kind	Name	Arguments	What it does
class	BlockTable	class	Speeds, delays, p_gate and p_edge arrays.
method	BlockTable.p_save	(speed, at_gate, delay=0.0)	Save probability for a speed, gate/edge start and delay (interpolated).
method	BlockTable._edge_at	(s, d)	Bilinear interpolation in the edge table.
function	_cache_key	(cfg, policy, speeds, delays, episodes, seed)	Hash of everything that changes the table.
function	_policy	(cfg, policy)	The block policy: scripted center or a trained run.
function	build_block_table	(cfg, policy='center', speeds=None, delays=None, episodes=40, seed=...)	Measure the table on the Phase 1 env.
function	load_or_build_block_table	(cfg, verbose=True)	Load the cached table, or build and cache it.

src/slingpuck/primitives/sling.py

Frozen sling primitive: timings and success probability · 29 lines

Holds the sling timing (move to the band, fetch a puck, sling core, flight) and the Phase 2 success probability, from the config.

Kind	Name	Arguments	What it does
class	<code>SlingPrimitive</code>	class	Sling timings and success probability.
method	<code>SlingPrimitive.from_config</code>	classmethod (cfg)	Build from <code>cfg['match']</code> .
method	<code>SlingPrimitive.at_band_s</code>	property ()	Time from the arm at the band to the release: fetch the puck, then the sling core.

Policies

src/slingpuck/policies/asymmetric.py

Asymmetric actor-critic for Stable-Baselines3 PPO · 74 lines

The actor reads only `obs['policy']` (deployable sensors); the critic reads `obs['policy']` and `obs['privileged']` (sim truth and randomized physics). Only the actor is deployed. A test checks that changing the privileged input does not change the actor output and does change the value.

How it works

- SB3 builds the actor feature extractor first and the critic extractor second; `make_features_extractor()` uses this order. `AsymmetricMlpExtractor` lets the actor and the critic networks have different input sizes.

Kind	Name	Arguments	What it does
class	<code>KeySelectExtractor</code>	<code>class(BaseFeaturesExtractor)</code>	Concatenate the selected keys of a Dict observation.
method	<code>KeySelectExtractor.__init__</code>	(observation_space, keys=ACTOR_KEYS)	Feature size = sum of the selected keys.
method	<code>KeySelectExtractor.forward</code>	(observations)	Concatenate the selected keys.
class	<code>AsymmetricMlpExtractor</code>	<code>class(nn.Module)</code>	MlpExtractor with different input sizes for the actor and the critic.
method	<code>AsymmetricMlpExtractor.__init__</code>	(pi_dim, vf_dim, net_arch, activation_fn, device)	Actor MLP on the actor input size, critic MLP on the critic size.
method	<code>AsymmetricMlpExtractor.forward</code>	(features)	Run both networks.
method	<code>AsymmetricMlpExtractor.forward_actor</code>	(features)	Actor network.
method	<code>AsymmetricMlpExtractor.forward_critic</code>	(features)	Critic network.
class	<code>AsymmetricActorCriticPolicy</code>	<code>class(ActorCriticPolicy)</code>	SB3 ActorCriticPolicy with separate actor and critic inputs.
method	<code>AsymmetricActorCriticPolicy.__init__</code>	(observation_space, action_space, lr_schedule, **kwargs)	Check the Dict space; force separate feature extractors.
method	<code>AsymmetricActorCriticPolicy.make_features_extractor</code>	()	First call: actor keys; second call: critic keys.
method	<code>AsymmetricActorCriticPolicy._build_mlp_extractor</code>	()	Use <code>AsymmetricMlpExtractor</code> .

src/slingpuck/policies/scripted.py

Scripted goalkeeper baselines (Phase 1) · 27 lines

`CenterBlocker` always commands `pan = 0`; `HoldStart` never moves. Both have the SB3 `predict()` signature, so the eval code treats them like trained models.

Kind	Name	Arguments	What it does
class	<code>CenterBlocker</code>	class	Always command pan = 0.
method	<code>CenterBlocker.predict</code>	(obs, state=None, episode_start=None, deterministic=True)	Return action 0 for every observation.
class	<code>HoldStart</code>	class	Never move.
method	<code>HoldStart.predict</code>	(obs, state=None, episode_start=None, deterministic=True)	Return the previous action.

src/slingpuck/policies/sling_scripted.py

Scripted sling baselines (Phase 2) · 46 lines

SlideCenter slides the puck to x = 0 and pulls straight back. SlideCenterCorrect also uses the tracked pulled puck to shift it back to x = 0 in the correction step.

Kind	Name	Arguments	What it does
class	<code>SlideCenter</code>	class	Slide to x = 0, pull straight back, no correction.
method	<code>SlideCenter.__init__</code>	(pull_frac=0.8)	Set the pull fraction.
method	<code>SlideCenter.predict</code>	(obs, state=None, episode_start=None, deterministic=True)	Aim action at step 1, zero correction at step 2.
class	<code>SlideCenterCorrect</code>	class(SlideCenter)	SlideCenter plus a lateral correction from the camera.
method	<code>SlideCenterCorrect.__init__</code>	(config, pull_frac=0.8)	Read the board length and the correction range.
method	<code>SlideCenterCorrect.predict</code>	(obs, state=None, episode_start=None, deterministic=True)	At step 2, shift the puck back to x = 0.

src/slingpuck/policies/match_scripted.py

Scripted match selectors (Phase 3) · 96 lines

AlwaysBlock, GreedySling (sling whenever possible, never return), ReloadRule (the plan's baseline: sling only when the opponent reloads, detected from the hand; otherwise go back to the gate), TellRule (also slings while the opponent aims if the hand x shows a likely miss) and OracleReload (ReloadRule with the true opponent phase).

Kind	Name	Arguments	What it does
class	<code>_Base</code>	class	Common predict() for Box or Dict observations.
method	<code>_Base.predict</code>	(obs, state=None, episode_start=None, deterministic=True)	Call act() with the policy (and privileged) vector.
method	<code>_Base.arm</code>	(obs, s)	True if the arm is in the given state.
class	<code>AlwaysBlock</code>	class(_Base)	Never sling.
method	<code>AlwaysBlock.act</code>	(obs, priv)	Return block.
class	<code>GreedySling</code>	class(_Base)	Sling whenever possible.
method	<code>GreedySling.act</code>	(obs, priv)	Return sling.
class	<code>ReloadRule</code>	class(_Base)	Sling only while the opponent reloads (hand away from its band line).
method	<code>ReloadRule.__init__</code>	(config, margin_m=0.01)	Read the opponent band line and the margin.
method	<code>ReloadRule.reloading</code>	(obs, priv)	Hand away from the opponent band line.
method	<code>ReloadRule.safe</code>	(obs, priv)	Safe to sling = opponent reloading.
method	<code>ReloadRule.act</code>	(obs, priv)	Sling if safe and no puck is incoming, else block.

Kind	Name	Arguments	What it does
class	<code>TellRule</code>	<code>class(ReloadRule)</code>	Also sling during the opponent's aim when the hand x shows a likely miss.
method	<code>TellRule.__init__</code>	<code>(config, miss_threshold_m=0.006, margin_m=0.01)</code>	Set the miss threshold for the hand x.
method	<code>TellRule.safe</code>	<code>(obs, priv)</code>	Reloading, or the hand x shows a likely miss.
class	<code>OracleReload</code>	<code>class(ReloadRule)</code>	Uses the opponent's true phase (privileged).
method	<code>OracleReload.reloading</code>	<code>(obs, priv)</code>	True opponent phase is reload or idle.

Training

src/slingpuck/train/common.py

Shared training code · 158 lines

Env factories (goalkeeper with a '2d' or 'mujoco' backend, sling), pickleable factories for SubprocVecEnv, `load_trained_run()` (loads a model with either import path), the outcome-rate TensorBoard callback, and `train_ppo()/train_cli()`: one reproducible PPO run per seed with its run folder (config, params version, git hash, seed, checkpoints, final model).

Kind	Name	Arguments	What it does
function	<code>make_goalkeeper_env</code>	<code>(config, asymmetric, backend='2d')</code>	backend: "2d" (Fast2DPuckSim + ServoModel) or "mujoco" (MuJoCo board, puck and SO-101).
function	<code>goalkeeper_env_fn</code>	<code>(config, asymmetric, backend='2d')</code>	Pickleable env factory for SubprocVecEnv.
function	<code>make_sling_env</code>	<code>(config, asymmetric)</code>	SlingEnv, wrapped for the asymmetric policy if needed.
function	<code>sling_env_fn</code>	<code>(config, asymmetric)</code>	Pickleable env factory for SubprocVecEnv.
function	<code>load_trained_run</code>	<code>(run_dir, device='cpu')</code>	Load final_model.zip from a training run.
function	<code>load_train_config</code>	<code>(path)</code>	Read a train_*.yaml file.
function	<code>new_run_dir</code>	<code>(task, params_version, seed)</code>	Run id and folder: logs/<task>/<task>_<params>_s<seed>_<time>.
class	<code>OutcomeLoggerCallback</code>	<code>class(BaseCallback)</code>	Log the fraction of episode outcomes (info["outcome"]) per rollout to TensorBoard.
method	<code>OutcomeLoggerCallback.__init__</code>	<code>(prefix, outcomes=('save', 'goal', 'none'))</code>	Set the outcome names to count.
method	<code>OutcomeLoggerCallback.on_step</code>	<code>()</code>	Count outcomes of finished episodes.
method	<code>OutcomeLoggerCallback.on_rollout_end</code>	<code>()</code>	Log outcome rates to TensorBoard and reset the counts.
function	<code>train_ppo</code>	<code>(task, env_fn_factory, outcomes, config, train_cfg, seed)</code>	One reproducible PPO run.
function	<code>train_cli</code>	<code>(task, env_fn_factory, outcomes, default_train_config)</code>	Command-line entry point shared by the train_* scripts.

src/slingpuck/train/train_goalkeeper.py

Phase 1 training entry point · 10 lines

Calls `train_cli` for the goalkeeper (configs/train_goalkeeper.yaml).

src/slingpuck/train/train_sling.py

Phase 2 training entry point · 10 lines

Calls train_cli for the sling shot (configs/train_sling.yaml).

Evaluation and viewers

src/slingpuck/eval/stats.py

Confidence intervals · 28 lines

wilson_ci() for proportions and t_ci() for means across runs (training seeds).

Kind	Name	Arguments	What it does
function	wilson_ci	(successes, n, confidence=0.95)	Wilson score interval for a binomial proportion.
function	t_ci	(values, confidence=0.95)	Mean and Student-t interval across independent runs (e.g.

src/slingpuck/eval/save_rate_vs_speed.py

Phase 1 eval: save rate vs puck speed · 244 lines

Runs trained goalkeeper runs and the scripted baselines on the same shots (paired seeds), with stratified speed bins, and reports the save rate vs speed, vs paddle start offset, and the speed x offset grid (the time-to-cover table), with 95% CIs. --backend mujoco runs the same eval on MuJoCo.

PPO CI = union of a t interval across training seeds and a Wilson interval over one seed's shots (all seeds learned the same saturated policy, so the t interval alone had zero width).

Kind	Name	Arguments	What it does
function	run_episodes	(policy, config, asymmetric, speed_bins, episodes_per_bin, base_seed...)	Return list of dicts: speed, start_offset (fraction of pan range), saved.
function	in_bin	(value, bins, b)	True if a value is in bin b (last bin includes its upper edge).
function	rate_with_ci	(groups, select)	Save rate and 95% CI over the rows that 'select' accepts, for one policy.
function	summarize	(results, key, bins)	results: {policy: [rows per group]}.
function	summarize_grid	(results, speed_bins, start_bins)	Save rate per (speed bin, start-offset bin) for each policy.
function	plot	(speed_table, start_table, out_path, subtitle)	Two panels: save rate vs speed and vs start offset, with CI bands.
function	write_csv	(path, table)	Write a list of dicts to CSV.
function	main	()	Command line: evaluate runs and baselines, write tables, plot and metadata.

src/slingpuck/eval/hole_rate.py

Phase 2 eval: gate success rate and sim-vs-real parity · 345 lines

Policy mode: success rate vs shot count and by puck start position, for trained runs and baselines.

Parity mode: replays every logged shot many times in the sim and compares (a) success (Brier score, reliability diagram) and (b) the transit time from 2 cm past the band line to $y = -4$ cm. Success depends mostly on the lateral aim and cannot see a speed error; the transit time can.

Kind	Name	Arguments	What it does
function	run_shots	(policy, config, asymmetric, n_shots, base_seed)	Run one policy for n shots; return per-shot results.
function	summarize	(results, x_bins)	Success rates overall and by start position, with CIs.
function	_style	(ax)	Common axis style.

Kind	Name	Arguments	What it does
function	<code>plot_policies</code>	(results, table, x_bins, out_path, subtitle)	Running success vs shot count, and success by start position.
function	<code>_crossing_time</code>	(t, y, y_level)	First time y reaches y_level (linear interpolation between frames), or None.
function	<code>_transit</code>	(t, y, band_y)	Transit time from 2 cm past the band line to y = -4 cm.
function	<code>parity</code>	(shots_csv, versions, cfg_dir, repeats, seed)	Replay logged shots in the sim for each params version.
function	<code>parity_report</code>	(observed, obs_transit, preds, out_dir)	Brier score, reliability table, transit bias; parity plot.
function	<code>write_csv</code>	(path, rows)	Write a list of dicts to CSV.
function	<code>main</code>	()	Command line: policy mode or parity mode.

`src/slingpuck/eval/match_eval.py`

Phase 3 eval: match results · 181 lines

Plays every policy on the same matches and reports win, loss and time-out rates, the mean puck difference, slings, pucks conceded while away and the save rate, with 95% CIs. Supports a tell_strength sweep (--tell) and the hand ablation (--no-hand). Each policy keeps a fixed color.

Kind	Name	Arguments	What it does
function	<code>play</code>	(policy, env, n_matches, base_seed, tell)	Play n matches with one policy; return per-match stats.
function	<code>summarize</code>	(name, tell, rows)	Rates and means with CIs for one policy and tell value.
function	<code>color_for</code>	(name, names)	Fixed color per policy: scripted policies keep their slot in every chart; runs follow them.
function	<code>plot</code>	(table, out_path, subtitle)	Win rate and puck difference: dot plot, or lines for a tell sweep.
function	<code>main</code>	()	Command line: scripted policies and trained runs, tell sweep, hand ablation.

`src/slingpuck/eval/visualize_goalkeeper.py`

Top-down slow-motion goalkeeper viewer · 197 lines

Shows the true puck, the last camera frame, the Kalman estimate and the paddle (faded while locked), in a live window or as a GIF. Records frames through the env's substep_hook.

Kind	Name	Arguments	What it does
function	<code>load_policy</code>	(args)	Load a trained run or a scripted baseline.
function	<code>record_episode</code>	(env, policy, seed, options, frame_dt, hold_s=0.3)	Run one episode and return snapshot dicts, one per frame_dt of sim time.
function	<code>draw_static</code>	(ax, gk)	Draw the board, divider, band line and paddle arc.
function	<code>animate</code>	(episodes, gk, title, frame_dt, slowmo, save)	Animate recorded episodes (window or GIF).
function	<code>main</code>	()	Command line for the 2D viewer.

`src/slingpuck/eval/view_mujoco.py`

3D MuJoCo viewer · 183 lines

Runs MujocoGoalkeeperEnv with a trained run or a scripted policy. Live window with mjpgpython (required on macOS) or an offscreen GIF. The blue ring is the Kalman estimate; the small sphere is the last camera frame. Cameras: gate, side, top.

Kind	Name	Arguments	What it does
function	<code>add_markers</code>	(scene, env, last_cam)	Kalman estimate ring and last camera frame, drawn above the puck.

Kind	Name	Arguments	What it does
function	<code>episode_options</code>	(args, env, k)	Reset options from the command line (speed, start offset, delay).
function	<code>run_episode</code>	(env, policy, seed, options, on_frame, frame_dt, pre_launch_s)	Step one episode; call <code>on_frame(env, last_cam)</code> every <code>frame_dt</code> of sim time near the launch.
function	<code>live</code>	(args, env, policy)	Live MuJoCo window, paced to the slow-motion factor.
function	<code>save_gif</code>	(args, env, policy)	Render episodes offscreen and save a GIF with a text header.
function	<code>main</code>	()	Command line for the 3D viewer.

System identification

src/slingpuck/sysid/schema.py

Log schemas · 64 lines

Column definitions for `band_bench`, `servo_step` and `shots` logs, `load_log()` (validates columns, empty values and phase names) and `find_logs()` (finds one log of each kind in a folder).

Kind	Name	Arguments	What it does
class	<code>LogError</code>	<code>class(ValueError)</code>	Error raised for a log that does not match its schema.
function	<code>load_log</code>	(path, kind)	Load and validate one log CSV.
function	<code>find_logs</code>	(log_dir)	Return {kind: path} for the log files present in <code>log_dir</code> (<kind>*.csv).

src/slingpuck/sysid/synthetic.py

Synthetic logs with known true values · 156 lines

Generates the three log types from the simulator with a 'truth' config that differs from `v0` (camera noise, dropouts, gauge noise, encoder quantization), plus `truth.yaml`, to test that the fits recover known values before real data exists.

Kind	Name	Arguments	What it does
function	<code>truth_config</code>	(base, truth)	Copy of the base config with the true values set.
function	<code>band_bench_log</code>	(cfg, rng, cycles=3, points=25)	Synthetic force-gauge cycles (load and unload) with gauge noise.
function	<code>servo_step_log</code>	(cfg, rng, sample_dt=0.005, duration=0.6)	Synthetic step tests with encoder noise and 12-bit quantization.
function	<code>_frames</code>	(cfg, rng, positions_fn, t0, t1)	Camera frames (capture times, noisy positions) of a trajectory between <code>t0</code> and <code>t1</code> .
function	<code>shots_log</code>	(cfg, rng, n_gate=40, n_divider=25)	Shots near the gate (most succeed), plus off-center shots that bounce off the divider.
function	<code>generate</code>	(out_dir, base_version='v0', truth=None, seed=0, n_gate=40, n_divid...)	Write <code>band_bench.csv</code> , <code>servo_step.csv</code> , <code>shots.csv</code> and <code>truth.yaml</code> .
function	<code>main</code>	()	Command line for the generator.

src/slingpuck/sysid/fit.py

Physics fits · 387 lines

`fit_band_bench`: band stiffness, exponent, hysteresis (natural length measured and held fixed, because the bench data cannot separate it from stiffness and exponent).

`fit_servo_steps`: latency (1 ms grid), deadband (0.02 deg grid, no useful gradient), lag and max rate (least squares on all tests at once, simulated as one vector).

`fit_shots`: compliance (measured pull depth vs command), friction (one deceleration shared by every free sliding segment: exit, through the gate, back from the divider), wall restitution (divider bounces) and energy transfer (exit speeds vs the

band model).

Important

- Four problems were found on synthetic data and fixed: parameters that trade off, a deadband stuck at its start value, frames selected by the noisy y (biased speeds 8% low; now a second pass by the fitted y), and a bounce between two frames that put post-bounce frames into the first segment. All shot parameters are now within ~1 standard error over 6 seeds.

Kind	Name	Arguments	What it does
class	<code>FitValue</code>	<code>class</code>	Fitted value, standard error, number of data points, note.
method	<code>FitValue.as_dict</code>	<code>()</code>	Plain dict for YAML.
function	<code>_stderr</code>	<code>(result, n_params)</code>	Linearized standard errors from a scipy least_squares result.
function	<code>with_values</code>	<code>(cfg, values)</code>	Copy of a config with fitted values set.
function	<code>fit_band_bench</code>	<code>(df, cfg)</code>	natural_length_m is taken from cfg (measure it directly) and held fixed.
function	<code>_servo_sim</code>	<code>(df, dt, latency_s, tau, rate_deg, deadband_deg)</code>	Simulate all step tests at once (vector ServoModel) for given parameters.
function	<code>fit_servo_steps</code>	<code>(df, cfg, dt=0.001, max_latency_s=0.15)</code>	Latency (1 ms grid) and deadband (0.02 deg grid) by search; lag and rate by least squares.
class	<code>_Shot</code>	<code>class</code>	Rest position, pulled position, flight frames and commands of one logged shot.
function	<code>_shots</code>	<code>(df)</code>	Group the shots log into _Shot objects.
function	<code>_path</code>	<code>(t, p, theta, s0, a)</code>	Straight sliding path from p at speed s0 (time t >= 0), constant deceleration a, stops at 0.
class	<code>_Segment</code>	<code>class</code>	Frames of one free sliding segment (exit, through, return).
function	<code>_raw_segments</code>	<code>(i, shot, band_y, face_y, half_length, radius)</code>	Free sliding segments of one shot, cut by the measured y (first pass).
function	<code>_seg_params_guess</code>	<code>(seg)</code>	Start values (position, heading, speed) for a segment.
function	<code>_fit_segments</code>	<code>(segs, a0)</code>	Shared deceleration + per segment (x, y at t0, heading, speed at t0).
function	<code>_reselect</code>	<code>(segs, fit, shots, band_y, face_y, half_length, radius, margin=0.006)</code>	Second pass: keep frames by the *fitted* y with a margin larger than the camera noise.
function	<code>_time_at_y</code>	<code>(px, py, th, s0, a, y_target)</code>	Time (relative to t0, may be negative) when the path reaches y_target, or None.
function	<code>fit_shots</code>	<code>(df, cfg)</code>	Compliance, friction, wall restitution and energy transfer from the shots log.

src/slingpuck/sysid/run_sysid.py

Sysid command · 131 lines

Runs the fits for the logs present and writes physics_params_v<N+1>.yaml: the parent with the fitted values, those values removed from 'placeholders', servo values in its servo: section, and a sysid: record (parent, log checksums, git hash, fits with standard errors, suggested +-2 sigma randomization ranges). Synthetic fits are never written to configs/.

Kind	Name	Arguments	What it does
function	<code>_sha256</code>	<code>(path)</code>	Short checksum of a log file.
function	<code>run</code>	<code>(log_dir, base_version='latest', out_dir=None, natural_length=None,...)</code>	Run the fits and write the new params version; return its path.
function	<code>main</code>	<code>()</code>	Command line for sysid.

Deployment (old code, M5)

src/slingpuck/deploy/export_onnx.py

ONNX export (old code, M5) · 44 lines

Exports the actor of an SB3 PPO model to ONNX. Written before the rework; M5 must update it for the asymmetric (Dict-observation) policy.

Kind	Name	Arguments	What it does
function	<code>export_ppo_to_onnx</code>	(model_path, onnx_path, obs_shape)	Exports a Stable-Baselines3 PPO policy to ONNX format.

src/slingpuck/deploy/latency_benchmark.py

ONNX latency benchmark (old code, M5) · 43 lines

Times ONNX inference on CPU (mean and 99th percentile) against the 30 Hz control period. Written before the rework; M5 must update the observation shapes and run it on the Jetson.

Kind	Name	Arguments	What it does
function	<code>benchmark_onnx_latency</code>	(onnx_path, obs_shape, num_iterations=1000)	Benchmarks CPU inference latency for the ONNX policy.

src/slingpuck/deploy/lerobot_interface.py

LeRobot interface stub (M5) · 95 lines

Control-loop stub for the real SO-101: read sensors, run the ONNX actor, send a rate-limited joint command. It already uses the shared GoalkeeperGeometry mapping. Its observation is still the old 6-value layout (FIXME for M5); the hardware calls are commented out.

Kind	Name	Arguments	What it does
class	<code>S0101Controller</code>	class	Control loop stub for the real arm.
method	<code>S0101Controller.__init__</code>	(onnx_path, config)	Load the ONNX actor, the shared geometry and the limits.
method	<code>S0101Controller.get_observation</code>	()	Polls camera tracker and hardware encoders.
method	<code>S0101Controller.write_action</code>	(action_vector)	Maps raw policy action to rate-limited hardware commands.
method	<code>S0101Controller.run_loop</code>	()	Sense, infer, act, sleep to keep the control rate.

9. Tests

127 tests, all passing (`pytest -q`, about 40 s). They cover the physics sanity checks the plan asked for (energy never increases through a bounce, the hysteresis loop closes, latency shifts signals correctly) and Gymnasium API compliance for every env.

Test file	Lines
<code>test_asymmetric_policy.py</code>	27
<code>test_band.py</code>	99
<code>test_camera.py</code>	71
<code>test_config.py</code>	77
<code>test_goalkeeper_env.py</code>	140
<code>test_kinematics.py</code>	44
<code>test_match.py</code>	182
<code>test_mujoco_goalkeeper.py</code>	119
<code>test_puck_dynamics.py</code>	168
<code>test_servo.py</code>	68
<code>test_sling_env.py</code>	78
<code>test_stats.py</code>	17
<code>test_sysid.py</code>	103
<code>test_tracker.py</code>	62

- **test_config**: versions, latest, strict mode, servo overrides, randomization, run records.
- **test_servo / test_camera / test_tracker**: exact latency, rate limit, lag, deadband; frame rate, latency shift, dropouts, noise; convergence, latency compensation.
- **test_puck_dynamics**: friction stop, restitution, energy never increases, gate and divider, puck-puck, paddle contacts.
- **test_kinematics / test_goalkeeper_env / test_asymmetric_policy / test_stats**: mapping, check_env, outcomes, privileged isolation, CIs.
- **test_mujoco_goalkeeper**: paddle on the 2D arc, friction, calibrated restitution, gate events, check_env.
- **test_band / test_sling_env**: closed hysteresis loop, release energy, deflection, straight launch, Phase 2 env.
- **test_sysid**: schema checks, recovery of known values from synthetic logs, versioned output, synthetic guard.
- **test_match**: tell behavior, puck conservation, rewards, sling timing, hand ablation, reload detection.

10. How to run

Run from the project root with the miniconda Python (python3); the package is imported as `src.slingpuck`. On macOS the live MuJoCo window needs `mjpython`.

```
pytest -q                                # all tests

# Phase 1
python -m src.slingpuck.train.train_goalkeeper          # 3 seeds
python -m src.slingpuck.eval.save_rate_vs_speed --runs logs/goalkeeper/goalkeeper_v0_s*_*
python -m src.slingpuck.eval.save_rate_vs_speed --runs logs/goalkeeper/goalkeeper_v0_s*_* --backend mujoco
python -m src.slingpuck.eval.visualize_goalkeeper --run logs/goalkeeper/<run>
mjpython -m src.slingpuck.eval.view_mujoco --run logs/goalkeeper/<run>

# Phase 2
python -m src.slingpuck.train.train_sling
python -m src.slingpuck.eval.hole_rate --runs logs/sling/sling_v0_s*_*

# Sysid (try it on synthetic logs)
python -m src.slingpuck.sysid.synthetic --out data/real_logs/synthetic
python -m src.slingpuck.sysid.run_sysid --logs data/real_logs/synthetic --natural-length 0.152
python -m src.slingpuck.eval.hole_rate --parity data/real_logs/synthetic/shots.csv \
    --versions v0 v1 --cfg-dir data/real_logs/synthetic/fit

# Phase 3
python -m src.slingpuck.eval.match_eval
python -m src.slingpuck.eval.match_eval --policies greedy_sling reload_rule tell_rule --tell 0 0.5 1

# This report
python docs/build_report.py
```

11. Constraints and difficulties faced

The limits and problems met during the project, and how each one was handled.

Constraint	Effect on the project and how it was handled
No real hardware data	There were no measurements from the real SO-101, board or band. 33 values are placeholders; all results are simulation results and must be repeated with measured values. Sysid was tested only on synthetic logs.
Incomplete and inconsistent board data	Board sizes came from an Amazon listing; the 'size M' text and the 'size S' image disagree. The gate width is only known as 'a little over 1.19 in'. The measured puck mass (27 g) gives an implausible density for wood and may be the mass of 6 pucks.
Unknown game rules and timings	The scoring rules, the robot's fetch/sling/move times and the human opponent's timing are unknown. Phase 3 results depend strongly on them, so they could only be shown as a regime map, not as an answer.
CPU-only laptop	All training ran on a laptop CPU (no GPU). Phase 1 PPO took 15-40 min per 1M-step seed (slower when other jobs shared the CPU). Background jobs in this session stop after a time limit; one 3-seed run was stopped after seed 1 and seed 2 had to be restarted.
Short episodes	On the real-size board a puck reaches the gate in ~100 ms (about 3 control steps), so most training time is spent in reset (pre-launch period, threat sampling), not in learning.
A very narrow gate	The gate is ~1.5 mm wider than the puck. This made 'go to center' optimal for a goalkeeper with time, forced the recovery-task design, and makes Phase 2 a +-3 mm aiming problem.
Physics of the band	With a pre-stretched band and short pulls, the pull angle cannot aim the puck. The plan's 2-D action had to be changed to include placement.
MuJoCo contact behavior	A free puck tumbled; contact friction on a soft floor made the puck hop; the (timeconst, dampratio) contact form was unstable for low damping at a 1 ms step (a 1.5 m/s hit came back at 6.4 m/s). Restitution also depends on the impact phase. Solved with a planar puck, an applied friction force, direct stiffness/damping, a 0.5 ms step and calibration in the real scene.
Arm geometry	The SO-101's pitch joints move the fingertips in a plane ~1 mm beside the pan axis, so the inverse kinematics cannot reach the exact target; a custom paddle mount absorbs the offset. The real end effector and its mount are not known.
Camera at 60 Hz with 50 ms latency	A flight to the gate lasts ~70 ms, so only ~5 frames per segment are available for sysid and the tracker barely sees a fast shot before it arrives. This limited the tracker and caused a selection bias in the first flight fit.
Parameters that cannot be separated	Band stiffness, exponent and natural length trade off in the bench data; hysteresis and energy transfer both scale the launch speed; camera latency needs a sync event. These need extra measurements (ruler, bench test, LED).
Software environment	Two Pythons on the machine (Homebrew 3.10 as the 'python' alias, without scipy and pytest; miniconda 3.14 with everything). The package import path changed to src.slingpuck during the project, and saved models store their class path, which needed a custom loader. The live MuJoCo window needs mpython on macOS. No display was available in the session, so live windows could not be tested; GIFs were used. reportlab was not installed, so this PDF is printed with headless Chrome.
Old code	The original code ran but had invalid results (sign error, missing goals, mapping mismatch, stale frames, ground-truth fallback, unseeded randomness). M1 had to rebuild the base before new work.

12. What can still be improved

Improvements to the parts that already exist (milestone M5 is in the next section).

Priority	Improvement	Why
High	Measure the real pan step response and replace the first-order ServoModel with a second-order, torque-limited model (or train Phase 1 in MuJoCo)	The 2D/MuJoCo gap shows that the servo model decides the Phase 1 result; PPO's 2D advantage did not transfer.
High	Get the real scoring rules and robot/human timings, then re-run Phase 3	The Phase 3 conclusion (a throughput race) may change completely.
High	Run sysid on real logs (band bench, servo steps, shots) and use the fitted version for all training	Every number in this report uses placeholders.
Medium	Add launch (maneuver) detection to the Kalman tracker and a bounce-aware prediction	The estimate is ~10 cm behind 80 ms after a launch; Phase 3 decisions need a good early estimate.
Medium	Speed up Phase 1 training (shorter pre-launch, fast-forward the locked period, cheaper threat sampling) and stop earlier	Training plateaus at ~50k of 1M steps.
Medium	Add pan-encoder noise and quantization to the observations; randomize servo rate and lag in MuJoCo	Closer to the real sensors; robustness.
Medium	Phase 2: model the puck radius in the band geometry and lateral arm deflection; measure the noise values with ~30 repeated shots	The miss rate is set by these values.
Medium	Phase 3: replace the fixed fetch time with puck positions; resolve several pucks at the gate; build the block table from the trained goalkeeper	Higher fidelity of the match model.
Low	Add a scripted 'overshoot' baseline to the Phase 1 eval	Explains PPO's 2D gain as a baseline.
Low	PyBullet backend (the interface is ready), if still wanted	MuJoCo already covers the arm.
Low	Packaging: one Python environment, a lock file, CI that runs the tests	Fewer setup problems.

13. What is still to do

Milestone M5

- Train the Phase 3 selector (PPO on MatchEnv, asymmetric critic), several seeds; compare with the rule-based baselines.
- Ablations: without hand kinematics (`--no-hand`), and a sweep over `tell_strength`; report how much reaction time the anticipation recovers.
- Vulnerability-window analysis (rebuild `vulnerability_window.py` on MatchEnv): risk of a goal vs the time of the sling decision relative to the opponent cycle.
- Deployment: update the ONNX export for the asymmetric actor, run the latency benchmark on the Jetson Orin Nano, and finish the LeRobot interface (8-value observation, real tracker, hardware calls, rate and torque limits).
- Run M5 either after the real values below are known, or as a sweep over them (user to decide).

Real measurements needed (all are placeholders now)

Measurement	Why it matters
Game scoring rules	Decide whether Phase 3 is a throughput race; the reward and win rule
Pan servo step response with the paddle mounted	Decides which servo model (2D or MuJoCo) is right; Phase 1 reaction time
Robot fetch, sling and move times	Length of the vulnerability window in Phase 3
Human opponent timing and accuracy (video)	Opponent cycle, threat rate, tell
Gate width, divider thickness, wall height	Gate geometry (Phase 1 threat zone, Phase 2 target)
One puck's mass; puck size on size M	Launch speed; all contacts
Band: peg spacing, distance to the end wall, max pull, natural length; force-gauge test	Band model fit
Shot log (incl. divider bounces) and shot repeatability	Compliance, friction, restitution, energy transfer; Phase 2 miss rate
Arm base position and height, joint zero offset, end-effector size and mass	Paddle arc, MuJoCo arm pose
Camera latency, noise, dropout rate	Tracker accuracy

Known limitations

- The Kalman tracker does not model wall bounces or a launch from rest (lags ~10 cm at +80 ms).
- Phase 1 training is slow per step because episodes are short; the pan encoder has no noise yet.
- MuJoCo: the threat check uses the 2D sim; servo rate and lag are not randomized.
- Phase 2: the band vertex is at the puck center; only deflection along the pull is modeled; camera latency and paddle restitution cannot be fitted from the current logs; the arm's reach to the band must be checked.
- Phase 3: outcomes come from tables, not physics; one opponent puck at a time; fixed fetch time; the opponent never blocks.
- Deployment code is from before the rework (M5).

14. Glossary

sim2real2sim	Train in sim, test on the real robot, use real logs to correct the sim (sysid), and train again.
Placeholder	A config value that is not measured yet; listed under <code>placeholders:</code> ; strict mode refuses it.
Domain randomization	Sampling physics values from ranges at every episode, so a policy does not depend on one exact value.
Asymmetric actor-critic	The actor (deployed) sees only real sensors; the critic (training only) also sees sim truth.
Privileged observation	Sim-only information (true positions, randomized values, opponent intent) given to the critic.
Restitution	Ratio of normal speed after and before a bounce (1 = no energy loss).
Hysteresis (band)	The band pulls back with less force than it took to stretch it; the loop area is the lost energy.
Compliance	How far the arm gives way per newton of band force (m/N).
Energy transfer	Fraction of the band's release work that the puck keeps as kinetic energy.
Recovery task / time to cover	Phase 1 setup where the paddle starts away from the gate; measures how fast the gate can be covered.
Frozen primitive	A trained or scripted low-level skill that Phase 3 uses as a fixed block (here: its measured outcome table).
Tell / tell_strength	How much the opponent's hand reveals where the shot will go (0 nothing, 1 exactly).
Vulnerability window	The time the gate is undefended because the arm is slinging or returning.
Wilson / t interval	95% confidence intervals for a proportion / for a mean across seeds.
Brier score	Mean squared error between predicted probabilities and 0/1 outcomes (lower is better).
Parity	Agreement between sim predictions and real logs for the same commands.